A thirty-attack cross-layer dataset and hybrid machine-learning intrusion detection for IoT-enabled wireless sensor networks

Abstract Internet of Things (IoT)-enabled Wireless Sensor Networks (WSNs) combine resource-constrained motes, multi-hop routing and Internet-facing services, so an attack on one layer can affect the others. However, most intrusion detection studies consider only a few routing attacks or rely on benchmark data that do not represent low-power networks. A 21-mote Sky network running Contiki OS 2.7 was simulated in Cooja, and thirty attacks, ten at each of the perception, network and application layers, were implemented as custom firmware on three malicious motes, alongside a normal baseline. The packet captures formed a 31-class dataset of 1,808,427 records with six packet-level features, split into stratified 70:30 training and test sets. Sixteen runs covering fifteen hybrid architectures (averaged, stacked and convolutional–recurrent hybrids) were compared using accuracy, macro-averaged precision, recall, F1-score, specificity, false alarm rate (FAR), class-level results and computational cost. The Random Forest + Extreme Gradient Boosting hybrid performed best, with 85.91% accuracy, 84.52% macro F1-score, 99.49% macro specificity and 0.51% macro FAR, and detected every class at least partly. Stacking improved the macro F1-score of an Extremely Randomized Trees + LightGBM pair by 6.18 percentage points over equal averaging. The simulations showed distinct attack footprints: channel-occupying attacks raised the power of legitimate motes up to 21-fold, whereas several application-layer attacks left no energy trace. Reactive jamming, which overlaps with legitimate traffic at the packet level, remained the main weakness. The selected detector forms the analytical core of a layered framework that links traffic collection, cross-layer monitoring, threat classification, decision-making, response and resilience.

Introduction

The Internet of Things (IoT) extends computing to everyday objects that sense, process and exchange information about the physical world, and Wireless Sensor Networks (WSNs) remain one of its most important sensing infrastructures [1, 2]. In an IoT-enabled WSN, battery-powered motes with limited memory and processing capacity collect measurements, relay them over multi-hop low-power links and deliver them to gateways, cloud platforms and applications. IPv6 connectivity and the Routing Protocol for Low-Power and Lossy Networks (RPL) make this practical by organizing motes into a Destination-Oriented Directed Acyclic Graph (DODAG) rooted at a border router [3]. The openness that makes these networks attractive for smart cities, healthcare, agriculture and industrial monitoring also exposes them to attacks on the radio channel, on routing and on application services [4, 5].

Security threats in IoT-enabled WSNs are commonly analyzed according to the three-layer IoT architecture of perception, network and application [6, 7]. At the perception layer, jamming, collisions, spoofed link-layer identities and energy depletion target the shared radio medium and the batteries of the motes [8, 9]. At the network layer, sinkhole, blackhole, selective-forwarding, Sybil, wormhole, hello-flood, rank and version-number attacks manipulate RPL control information and forwarding paths [10–12]. At the application layer, falsified or forged data, replayed messages, denial of service (DoS), hijacked firmware-over-the-air (FOTA) updates and weak authentication undermine the services that consume sensor data [13–15]. These layers are not independent: a captured mote can advertise a false route, attract traffic and then inject falsified readings, and an application-layer flood can exhaust the energy of the motes that must relay it [16]. Table 1 summarizes the attack families considered in this study, their primary targets and the security properties they affect.

Table 1. Attack families simulated at each layer of the IoT-enabled WSN.

Layer	Simulated attacks (ten per layer)	Primary targets	Main security properties affected
Perception (physical/MAC)	Constant, deceptive, reactive and random jamming; energy exhaustion; collision; MAC spoofing; de-synchronization; eavesdropping; unfairness	Radio channel, transceiver and duty cycle; link-layer identity; mote battery	Availability, energy, authenticity, confidentiality
Network (RPL routing)	Sinkhole; blackhole; selective forwarding (grayhole); Sybil; wormhole; hello flood; decreased rank; increased rank; version number modification; routing replay	Parent selection; rank and version fields; DIO control traffic; forwarding paths; node identity	Integrity, availability, freshness, authenticity
Application (data and services)	Data falsification; message forgery; malicious code injection; buffer overflow; application-level DoS; FOTA hijacking; brute force; repudiation; time synchronization; data replay	Sensor payloads; sink services; firmware updates; credentials; audit records; timestamps	Integrity, authenticity, accountability, availability

Intrusion Detection Systems (IDSs) based on Machine Learning (ML) and Deep Learning (DL) are widely proposed as the adaptive layer of IoT security [5, 17]. Many of these systems are still trained and tested on general-purpose benchmarks such as NSL-KDD and CICIDS2017, whose traffic does not reflect RPL control messages, duty-cycled radios or constrained motes [18–20]. For this reason, simulation with the Contiki operating system and its Cooja network simulator has become a common way to generate IoT-specific data under controlled and repeatable conditions [21, 22]. Recent Cooja-based studies report high detection rates for sinkhole, version-number, flooding, blackhole and rank attacks [23–26]. Most of these datasets, however, contain between one and five attack types, concentrate on the routing layer and are evaluated as binary or few-class problems, often with accuracy as the headline metric. Cross-layer security frameworks, for their part, describe how trust, detection and response should be coordinated across layers [27–29], but they rarely provide an empirical detector that has been tested against attacks from all three layers at once.

This study addresses that gap by combining large-scale cross-layer attack simulation with a systematic comparison of hybrid ML detectors and by embedding the selected detector in a layered security framework. A 21-mote RPL network running Contiki OS 2.7 was simulated in Cooja, and thirty attacks, ten at each layer, were implemented as custom firmware on three malicious motes. The resulting 31-class dataset of 1,808,427 packet records was used to train and compare sixteen experimental runs representing fifteen hybrid architectures, ranging from probability-averaged tree ensembles to a stacked ensemble and convolutional–recurrent networks. The evaluation deliberately goes beyond accuracy and reports macro-averaged precision, recall, F1-score, specificity, false alarm rate (FAR), class-level results and computational cost.

Problem statement

Existing protection for IoT-enabled WSNs is fragmented. Encryption and authentication are usually applied at the application layer, whereas intrusion detection is mostly studied for RPL routing attacks in isolation, and datasets that capture perception-, network- and application-layer attacks under one consistent experimental setup are scarce. As a result, it is unclear how well a single detector can discriminate a broad and imbalanced set of cross-layer attacks, which hybrid learning strategy is most effective for this task and how such a detector should feed decision-making, response and resilience in a resource-constrained network. A layered, empirically grounded security framework is needed that detects threats across the whole IoT stack and connects detection to defensive action.

To address these gaps, the study pursued four objectives: (i) to classify the security threats that affect each layer of IoT-enabled WSNs; (ii) to identify the limitations of existing IoT security frameworks; (iii) to design a layered security model for IoT-enabled WSNs; and (iv) to verify the detection core of this model through simulation. The main contributions of the study are as follows:

- A cross-layer attack corpus in which thirty attacks (ten perception-, ten network- and ten application-layer attacks) are implemented as executable Cooja/Contiki scenarios with the same 21-mote RPL configuration, together with a normal baseline
- A labeled 31-class dataset of 1,808,427 packet records with documented preprocessing and with power, duty-cycle and packet-delivery evidence for every scenario
- A controlled comparison of sixteen experimental runs (fifteen hybrid architectures) on identical data partitions, reported with macro-averaged metrics, class-level results, confusion matrices and computational cost
- Evidence that the fusion rule strongly affects hybrid IDS performance and that deeper convolutional–recurrent designs do not guarantee better multiclass detection
- A layered security framework that places the selected hybrid detector at its analytical core and links traffic collection, cross-layer monitoring, threat classification, decision-making, response and resilience, with a clear separation between validated and proposed components

Table 2 summarizes representative intrusion detection studies for IoT and WSNs that are closest to this work.

The research gaps addressed in this study are as follows:

- **Narrow attack coverage:** Cooja-based datasets typically contain one to five routing attacks [23, 26], so perception- and application-layer behavior is rarely represented. This study simulates thirty attacks spread evenly across three layers.
- **Binary or few-class evaluation:** Separate normal-versus-attack models per scenario face an easier task than a single detector that must separate thirty attacks and normal traffic. This study evaluates one 31-class problem.

Table 2. Summary of representative intrusion detection studies for IoT and WSNs.

Reference	Approach adopted	Detection model(s)	Dataset and attacks	Reported performance
Kamel and Elhamayed [30]	Three-layer IoT-healthcare framework that predicts routing attacks and their power impact, with SMOTE and feature selection	CNN	Cooja-simulated hello flood, selective forwarding, sinkhole, wormhole and version attacks	Per-attack accuracy 90.40–98.57% (as summarized by Choudhary et al. [23])
Liu et al. [20]	NSL-KDD attack patterns replayed in a Contiki-NG network; eleven learners compared	XGBoost, RF, DT, bagging, SVM, naïve Bayes, AdaBoost and unsupervised methods	NSL-KDD-based traffic; 7 classes (flooding, DoS and scanning)	Accuracy 97.0%, MCC 90.5% (XGBoost)
Wan et al. [31]	Multi-layer (system, network and application) monitoring on programmable home routers	Supervised and unsupervised learning	Real smart-home traffic	Precision 0.9876; recall 0.9763
Momand et al. [32]	ML-assisted secure RPL routing from Cooja traffic	SVM with improved PCA	Cooja; version, rank and DoS attacks (30 nodes)	PDR 76.8% with 1,474 control packets
Ullah et al. [33]	Hybrid DL with recursive feature elimination	CNN–GRU	BoT-IoT, IoT-23, MQTT-IoT-IDS2020 and IoT Network Intrusion; up to 15 attacks + normal	Multiclass accuracy up to 99.92%
Rabhi et al. [34]	One normal and three malicious RPL scenarios simulated and captured	WEKA classifiers	Cooja; three RPL attacks	Precision above 96%
Choudhary et al. [23]	IoT-specific anomaly datasets generated in Cooja; CNN tuned with the Aquila optimizer	CNN-AO, CNN, LSTM	Cooja; sinkhole, version number and flooding (one binary dataset each)	Accuracy 99.22%, 99.77% and 99.55%
Farea and Küçük [24]	KoU-6LoWPAN-IoT dataset from Contiki/Cooja; hybrid intrusion detection and prevention	ANN and decision-tree models	KoU-6LoWPAN-IoT (4 classes); Edge-IIoT (15 classes)	Accuracy 99.84% (KoU) and 97.64% (Edge-IIoT)
Khan et al. [35]	Feed-forward neural model of WSN routing behavior	ANN	CICIDS2017 (NS2); blackhole, grayhole, wormhole + normal	Accuracy 99.49%; detection rate 99.21%
Vora et al. [36]	ML IDS for Contiki-OS IoT validated in simulation and on hardware	AdaBoost, RF	Edge-IIoTset; DDoS, sinkhole and MITM	Accuracy 91.08%
Soumya et al. [25]	Blackhole attack simulated with and without a malicious node; traffic analysis	Autoencoder, Isolation Forest	Cooja/Contiki-NG; blackhole	PDR fell from 99% to 0.15%; accuracy about 92%
Upadhiyay and Jain [29]	Fog-assisted WSN framework joining energy-aware ACO routing and ensemble threat detection	SVM, KNN and LSTM ensemble; CatBoost/XGBoost-guided routing	Synthetic MATLAB data; DoS, probe, R2L and U2R	Accuracy above 90%; detection rate 89%
Panda and Muthuraman [26]	Simulation-driven multiclass RPL dataset (static and mobile); fifteen classifiers and ensembles	LR, SVM, DT, KNN, RF, ET, GB, AdaBoost, XGBoost; voting, stacking, bagging	Cooja/Contiki-NG; 30,240 samples; blackhole, decreased rank, version, flooding + benign	Accuracy 99.47% (soft voting)

ACO = ant colony optimization; ANN = artificial neural network; DT = decision tree; GB = gradient boosting; KNN = k-nearest neighbors; LR = logistic regression; MCC = Matthews correlation coefficient; MITM = man-in-the-middle; PCA = principal component analysis; PDR = packet delivery ratio; SMOTE = synthetic minority oversampling technique; SVM = support vector machine.

- **Accuracy-centered reporting:** With imbalanced classes, accuracy can hide poor detection of minority attack classes. This study reports macro metrics, class-level recall, confusion matrices, FAR and computational cost.
- **Detection disconnected from response:** Frameworks describe cross-layer defense conceptually, whereas detectors are evaluated in isolation [28, 37]. This study links an experimentally selected detector to a layered framework and states which components have been validated.

Materials and methods

The study followed an experimental and quantitative design organized in five phases (Fig 1): environment setup, attack simulation, dataset generation, hybrid model training, and evaluation with framework integration. The output of each phase served as the input to the next. The independent variable was the presence and type of malicious behavior, and the observed outcomes were packet-level traffic, routing structure, delivered and lost packets, power consumption and radio duty cycle.

Fig 1. Five-phase research methodology adopted in this study.

Ethics statement

All attacks were implemented and executed only in an isolated simulation environment; no real networks, devices or personal data were involved. The corresponding author confirms that this manuscript is original and has not been submitted elsewhere.

Simulation environment

All simulations were run on a laptop with an AMD Ryzen 5 4500U processor, 8 GB of RAM and a 256 GB SSD. Microsoft Windows 11 hosted an Ubuntu Linux virtual machine in which the Contiki OS 2.7 toolchain and the Cooja simulator were executed, and data analysis and model development were carried out in Python 3.12 notebooks in Google Colaboratory (Table 3).

Table 3. Hardware and software environment.

Component	Configuration
Host operating system	Microsoft Windows 11
Simulation environment	Ubuntu Linux virtual machine
IoT operating system	Contiki OS 2.7
Network simulator	Cooja
Simulation host processor	AMD Ryzen 5 4500U
Simulation host memory and storage	8 GB RAM; 256 GB SSD
Simulation host graphics	AMD Radeon Graphics
Data analysis and ML	Python 3.12 notebooks in Google Colaboratory with pandas, scikit-learn, XGBoost, LightGBM, CatBoost and TensorFlow/Keras

The network consisted of 21 emulated Sky motes running the Contiki IPv6/RPL collection example (Table 4). Node 1 acted as the RPL DODAG root and User Datagram Protocol (UDP) sink, and the remaining motes ran the UDP sender application. Radio propagation was modeled with the Unit Disk Graph Medium (UDGM) using a 50 m transmission range, a 100 m interference range and transmission and reception success ratios of 1.0. Every scenario used a fixed random seed (123456) and a 1 s mote start-up delay. Node positions were generated with Cooja's random positioning and stored in each scenario file, so the layout differs between scenarios; in every recorded network map, all motes lie inside the 50 m transmission

range of the root (S1–S31 Figs). RPL used the Minimum Rank with Hysteresis Objective Function (MRHOF) [38].

Table 4. Cooja simulation parameters and node roles.

Parameter	Value
Simulator and operating system	Cooja with Contiki OS 2.7
Mote platform and count	Sky mote; 21 motes
Root and sink	Node 1 (RPL DODAG root, UDP sink)
Legitimate senders	Nodes 2–21 (baseline); nodes 2–18 (attack runs)
Malicious motes	Nodes 19–21 (attack runs)
Routing protocol	IPv6 RPL
Objective function	MRHOF
Radio medium	UDGM (distance loss)
Transmission / interference range	50 m / 100 m
TX / RX success ratio	1.0 / 1.0
Random seed	123456
Mote start-up delay	1 s (1,000,000 µs)
Node placement	Random positioning, stored per scenario (layout differs between scenarios)

Simulation phase

Baseline and attack injection. In the baseline scenario, all twenty non-root motes (nodes 2–21) generated normal UDP collection traffic toward the sink; this run established the reference DODAG and the reference packet, routing and energy behavior. For each attack, the Cooja configuration was copied, the firmware of nodes 19–21 was replaced with the corresponding attack program, and the simulation was restarted with the same root and legitimate senders (nodes 2–18). Every attack scenario thus contained 21 motes, namely one sink, seventeen legitimate senders and three attackers, and was executed separately with its own configuration file, attack source, packet capture, comma-separated values (CSV) export and visual evidence. The attack programs differ in mechanism and timer frequency (Table 5). Where a physical-layer or software exploit could not be reproduced literally in Cooja, such as radio jamming, wormhole tunneling, buffer overflow or code injection, it was emulated through its observable network behavior; Table 5 describes the mechanism that the code actually executes so that the capability of the simulator is not overstated.

Perception-layer attacks. The perception-layer scenarios targeted the radio channel, medium access control (MAC) and the energy of the motes. The constant jammer multicast 60-byte garbage payloads to the all-nodes address every 50 ms, whereas the deceptive jammer sent realistic sensor readings at a similar rate so that its traffic looked valid at first glance. The reactive jammer polled the radio 50 times per second and transmitted a noise burst only when the channel was busy or a packet was being received, and the random jammer alternated pseudo-random sleep periods with bursts of 5–14 packets. Energy exhaustion amplified every send event into 50 protocol-conformant packets, the collision attack transmitted short bursts that contended with legitimate frames, MAC spoofing re-randomized the link-layer address every 2 s, and de-synchronization multicast false MAC timestamps. The eavesdropper disabled duty cycling and kept its radio in receive mode without injecting traffic, and the unfairness attack combined a permanently active radio with channel-hogging traffic at 50 packets/s.

Network-layer attacks. The network-layer scenarios manipulated the RPL state and packet forwarding. The sinkhole attacker forced its advertised rank to 256 every 5 s to appear close to the root; the blackhole attacker additionally removed its preferred parent so that attracted packets were dropped; and the selective-forwarding (grayhole) attacker switched randomly between dropping and forwarding. The decreased- and increased-rank attacks forced ranks of 256 and 10,000, respectively, and reset the DODAG Information Object (DIO) timer to advertise the change immediately. The version-number attack incremented the DODAG version every 10 s to trigger repeated global repairs, and routing replay periodically re-advertised stale rank and version information. Identity and path attacks were emulated through a cloned identity of

Table 5. Implementation of the thirty attack scenarios on malicious motes 19–21.

Attack	Implementation in Cooja (malicious firmware)	Trigger or rate	Primary security impact
Perception layer			
Constant (radio) jamming	60-byte garbage payload multicast to all nodes, keeping the channel occupied	20 packets/s	Availability; energy
Deceptive jamming	Legitimate-looking sensor payload (temperature and humidity values) multicast at a high rate	16 packets/s	Availability; energy
Reactive jamming	Channel polled; a noise burst is sent only when the channel is busy or a packet is being received	Channel checked 50 times/s	Availability; energy
Random jamming	Pseudo-random sleep followed by a burst of garbage packets	Sleep 1–5 s; bursts of 5–14 packets	Availability; energy
Energy exhaustion (battery drain)	Each normal send event amplified into 50 protocol-conformant UDP packets	50 packets per send event	Availability; network lifetime
Collision	Short collision bursts that contend with legitimate frames	20 packets/s	Integrity; availability; energy
MAC spoofing	Link-layer address re-randomized, followed by a ghost-node hello broadcast	Identity changed every 2 s	Authenticity; integrity
De-synchronization	False synchronization beacons carrying a fake MAC timestamp	4 packets/s	Availability; freshness; energy
Eavesdropping (sniffing)	Duty cycling disabled and radio held in receive mode; no packets injected	Continuous listening	Confidentiality; privacy
Unfairness	Duty cycling bypassed and channel-hogging multicast traffic	50 packets/s	Availability; fairness
Network layer			
Sinkhole	Advertised rank forced to 256 to attract neighboring traffic	Every 5 s	Integrity; availability; confidentiality
Blackhole	Low rank (256) plus removal of the preferred parent, so attracted packets are dropped	Every 5 s	Availability; delivery reliability
Selective forwarding (grayhole)	Low rank with random (about 50%) switching between dropping and forwarding states	Decision every 5 s	Integrity; availability
Sybil	Link-layer identity cloned from node 5; cloned-identity data multicast	Every 2 s	Authenticity; integrity
Wormhole	Low rank with tunnel-encapsulation multicast emulating a false short path	Every 2 s	Integrity; confidentiality; availability
Hello flood	High-rate routing-style broadcast to all nodes	25 packets/s	Availability; energy; topology integrity
Decreased rank	Rank forced to the minimum value (256) with immediate DIO timer reset	Every 5 s	Integrity; availability
Increased rank	Rank forced to 10,000 with DIO timer reset	Every 5 s	Availability; energy; topology stability
Version number modification	DODAG version incremented and DIO timer reset, triggering global repair	Every 10 s	Availability; energy
Routing replay	Stale RPL state re-advertised (rank 256, decremented DODAG version)	Every 6 s	Integrity; freshness; availability
Application layer			
Data falsification	Sensor fields overwritten (temperature 9999, humidity 0, light 9999) in otherwise valid reports	Normal sender schedule	Integrity; safety; trustworthiness
Message forgery	Fabricated critical alarm with a system-override command sent to the sink	Every 4 s	Authenticity; integrity
Malicious code injection	Executable-like payload with shell markers and a fake root command sequence sent to the sink	Every 5 s	Integrity; confidentiality; availability
Buffer overflow	Oversized payload with an overflow marker and padding sent to the sink	Every 3 s	Integrity; availability
Application-level DoS	Heavy application payload flooded to the IPv6 address of the sink	20 packets/s	Availability; energy
FOTA hijacking	Forged firmware-update block with update command, binary chunk and reboot instruction multicast	Every 4 s	Integrity; authenticity; device trust
Unauthorized access (brute force)	Authentication requests with incrementing password guesses sent to the sink	10 guesses/s	Confidentiality; integrity; authorization
Repudiation	Commands with stripped sender identifier and null audit trace	Every 5 s	Accountability; non-repudiation
Time synchronization	False application clock beacon (epoch shift of +999,999,999 s) multicast	Every 3 s	Integrity; freshness; availability
Data replay	Valid report whose sequence number is fixed at a stale value (5)	Normal sender schedule	Freshness; integrity; authorization

node 5 (Sybil), a low-rank node that multicast tunnel-encapsulated packets (wormhole) and a routing-style broadcast at 25 packets/s (hello flood).

Application-layer attacks. The application-layer scenarios kept the RPL paths legitimate and altered the content, rate or semantics of application traffic. Data falsification overwrote sensor fields with implausible values, message forgery sent a fabricated critical alarm, and malicious code injection and buffer overflow delivered executable-like and oversized payloads, respectively, to the sink. The application-level DoS attack flooded the sink with heavy payloads at about 20 packets/s per attacker, and FOTA hijacking multicast a forged firmware block with an update command and a reboot instruction. The brute-force attacker sent about ten password guesses per second, repudiation stripped sender and audit fields from commands, the time-synchronization attack broadcast an application clock shifted by +999,999,999 s, and data replay fixed the sequence number of otherwise valid reports at a stale value.

Instrumentation. Several observation tools were used in every run so that no attack was characterized by packet counts alone. The Cooja log listener recorded the execution messages of the malicious firmware; the network visualizer documented node positions and routing links; the PowerTracker and Collect View tools reported CPU, low-power mode (LPM), radio-listen and radio-transmit power together with duty cycles and per-node received and lost packets; the RPL network graph, labeled with the expected transmission count (ETX) of each link, documented parent relations; and the Radio Logger stored all radio traffic in packet capture (PCAP) format. Similar multi-level trace collection has been recommended for building Cooja-based IDS datasets [39].

Data generation and data preparation phase

The PCAP captures were converted into CSV packet records with seven fields: No., Time, Source, Destination, Protocol, Length and Info. Because every attack was executed in its own scenario, the ground-truth label of each record was taken from the scenario that produced it, and baseline records formed the Normal class; the labels were thus known by design rather than inferred from traffic. The scenario files were merged into a master dataset of 1,808,427 records in 31 classes (Fig 2).

Fig 2. Data generation and preprocessing pipeline from Cooja scenarios to model-ready training and test partitions.

The class distribution is markedly imbalanced (Fig 3). Normal traffic accounts for 338,668 records (18.73%), perception-layer attacks for 686,451 (37.96%), network-layer attacks for 428,515 (23.70%) and application-layer attacks for 354,793 (19.62%). The largest attack class (application-level DoS, 97,066 records) contains 14 times as many records as the smallest (eavesdropping, 6,921 records), and eleven attack classes each contribute less than 1% of the records.

Fig 3. Distribution of the 31 traffic classes in the master dataset (1,808,427 records).

The field No. is a record index and was excluded, leaving six predictors, $X = \{\text{Time}, \text{Source}, \text{Destination}, \text{Protocol}, \text{Length}, \text{Info}\}$, and the 31-class Label as the target (Table 6). Initial inspection found 141,685 records (7.83%) without Source and Destination values. These records were retained: the four categorical fields were converted to strings and stripped of surrounding white space, and missing entries were encoded explicitly as the category MISSING. The data were then divided into training and test partitions with a stratified 70:30 hold-out (random state 42), which gave 1,265,898 training and 542,529 test records with all 31 classes present in both partitions.

All transformations were fitted on the training partition only and then applied to the test partition to prevent leakage of test information. The numerical features Time and Length were standardized with the z-score in Eq (1), using the training mean and standard deviation (381.2902 and 330.4217 for Time; 65.8590 and 26.6542 for Length):

$$z = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}} \tag{1}$$

Table 6. Dataset attributes, roles and preprocessing.

Attribute	Type	Role	Preprocessing and statistics
No.	Integer	Record index (excluded)	–
Time	Float	Numerical predictor	z-score; training mean 381.2902, SD 330.4217
Source	String	Categorical predictor	Ordinal code; 772 categories; 141,685 missing values set to MISSING
Destination	String	Categorical predictor	Ordinal code; 38 categories; 141,685 missing values set to MISSING
Protocol	String	Categorical predictor	Ordinal code; 4 categories
Length	Integer	Numerical predictor	z-score; training mean 65.8590, SD 26.6542
Info	String	Categorical predictor	Ordinal code; 382 categories
Label	String	Target	Label encoding; 31 classes

The categorical features were ordinal-encoded with an encoder fitted on the training data, and categories unseen during training were mapped to -1; the training partition contained 772 Source, 38 Destination, 4 Protocol and 382 Info categories. The class labels were encoded as integers from 0 to 30. A final integrity check confirmed that both partitions had six predictors, all 31 classes and no missing or infinite values, after which the matrices and the fitted transformers were stored so that every model started from the same frozen representation.

Model training phase

Sixteen experimental runs were executed on the frozen partitions (Table 7). They represent fifteen distinct hybrid architectures, because Run 15 repeats the configuration of Run 6 as an internal reproducibility check. Three construction strategies were used (Fig 4). In twelve runs (Runs 1–6 and 11–16), two base learners were trained independently on the same preprocessed traffic, each learner produced a probability for each of the 31 classes, and the two probability vectors were averaged with equal weights, as in Eq (2); the predicted class was the one with the highest hybrid probability, as in Eq (3):

$$P_H(c \mid x) = 0.5\, P_1(c \mid x) + 0.5\, P_2(c \mid x) \tag{2}$$

$$\hat{y} = \arg\max_c P_H(c \mid x), \quad c \in \{1, \ldots, 31\} \tag{3}$$

Fig 4. Hybrid construction strategies evaluated in this study. (a) Equal-weight probability averaging, (b) stacking with a logistic-regression meta-learner and (c) sequential CNN–recurrent networks.

The component learners were Random Forest (RF) [40], Extremely Randomized Trees (ET) [41], Extreme Gradient Boosting (XGBoost) [42], the Light Gradient Boosting Machine (LightGBM) [43], Categorical Boosting (CatBoost) [44], Adaptive Boosting (AdaBoost) [45], Histogram-based Gradient Boosting (HistGB) and the Multilayer Perceptron (MLP). The pairings combine bagging with boosting (RF + XGBoost, ET + LightGBM), different boosting variants (XGBoost + LightGBM, HistGB + LightGBM, AdaBoost + CatBoost) and tree ensembles with a neural network (the MLP hybrids).

Run 10 used the same ET and LightGBM families as Run 3 but replaced fixed averaging with stacking [46]. The training partition was split 80:20 into 1,012,718 and 253,180 records; the base learners were fitted on the first part, and their concatenated class probabilities for the second part formed the 62-dimensional meta-feature vector $z = [P_{\mathrm{ET}}(x), P_{\mathrm{LGBM}}(x)]$ used by a logistic-regression meta-learner, as in Eq (4):

$$\hat{y} = \arg\max_c P_{\mathrm{LR}}(c \mid z) \tag{4}$$

Runs 7–9 were sequential deep hybrids that combined a convolutional neural network (CNN) with a recurrent network. In each of them, a one-dimensional convolutional layer with 64 filters (kernel size 3, ReLU

Table 7. Configuration of the sixteen experimental runs.

Run	Hybrid model	Construction	Main settings
1	AdaBoost + CatBoost	Soft voting (equal averaging)	AdaBoost: decision trees of depth 3, 50 estimators; CatBoost: 100 iterations, learning rate 0.1, depth 6
2	CatBoost + MLP	Soft voting (equal averaging)	CatBoost as in Run 1; MLP: one hidden layer of 50 units, 50 iterations, early stopping, standardized inputs
3	ET + LightGBM	50:50 probability averaging	ET: 50 trees, depth 12; LightGBM: 100 trees, learning rate 0.1, depth 6
4	RF + XGBoost	50:50 probability averaging	RF: 100 trees without a depth limit; XGBoost: 100 trees, learning rate 0.1, depth 6, multiclass log-loss
5	HistGB + LightGBM	50:50 probability averaging	HistGB: 100 iterations, learning rate 0.1, depth 6; LightGBM as in Run 3
6	XGBoost + LightGBM	50:50 probability averaging	XGBoost and LightGBM: 100 trees, learning rate 0.1, depth 6
7	CNN + GRU	Sequential deep hybrid	Conv1D(64, kernel 3, ReLU) → MaxPooling(2) → GRU(50) → Dropout(0.2) → Dense(31, softmax); Adam; batch 1,024; up to 10 epochs; early stopping
8	CNN + LSTM	Sequential deep hybrid	As in Run 7 with LSTM(50)
9	CNN + SimpleRNN	Sequential deep hybrid	As in Run 7 with SimpleRNN(50)
10	ET + LightGBM (stacking)	Stacking, logistic-regression meta-learner	Base split 1,012,718 / 253,180; ET (50 trees, depth 12) and LightGBM (100 trees, learning rate 0.1, depth 6)
11	ET + MLP	50:50 probability averaging	ET as in Run 3; MLP: hidden layers (64, 32), ReLU, Adam, 50 iterations, batch 256
12	LightGBM + MLP	50:50 probability averaging	LightGBM as in Run 3; MLP as in Run 11
13	XGBoost + MLP	50:50 probability averaging	XGBoost as in Run 4; MLP as in Run 11
14	RF + MLP	50:50 probability averaging	RF: 100 trees, depth 12; MLP as in Run 11
15	XGBoost + LightGBM (repeat)	50:50 probability averaging	Identical to Run 6; retained as a reproducibility check
16	RF + LightGBM	50:50 probability averaging	RF: 100 trees, depth 12; LightGBM as in Run 3

activation) and a max-pooling layer extracted local patterns from the six-feature input; a recurrent layer of 50 units followed, namely a Gated Recurrent Unit (GRU) [47], a Long Short-Term Memory (LSTM) network [48] or a simple recurrent network; and a dropout layer (rate 0.2) and a 31-way softmax layer produced the output. These models were trained with the Adam optimizer, a batch size of 1,024, up to 10 epochs, 10% of the training data for validation and early stopping with a patience of two epochs. All models were implemented with scikit-learn [49], XGBoost, LightGBM, CatBoost and TensorFlow/Keras, and the tree-based models used a random state of 42.

Evaluation phase

Every model was evaluated on the same independent test partition of 542,529 records. For each class c, the true positives (TP_c), false positives (FP_c), false negatives (FN_c) and true negatives (TN_c) were obtained in a one-versus-rest manner from the 31×31 confusion matrix, and the following metrics were computed. **Accuracy** is the proportion of all N test records that are classified correctly:

$$\mathrm{Accuracy} = \frac{\sum_{c=1}^{31} \mathrm{TP}_c}{N} \tag{5}$$

Precision is the proportion of records predicted as class c that truly belong to it, and **recall** (sensitivity) is the proportion of records of class c that are detected:

$$\mathrm{Precision}_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FP}_c} \tag{6}$$

$$\mathrm{Recall}_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FN}_c} \tag{7}$$

The **F1-score** is the harmonic mean of precision and recall:

$$\mathrm{F1}_c = \frac{2 \times \mathrm{Precision}_c \times \mathrm{Recall}_c}{\mathrm{Precision}_c + \mathrm{Recall}_c} \tag{8}$$

Specificity measures how reliably records of the other classes are kept out of class c, and the false alarm rate is the proportion of records of the other classes that are wrongly assigned to class c:

$$\mathrm{Specificity}_c = \frac{\mathrm{TN}_c}{\mathrm{TN}_c + \mathrm{FP}_c} \tag{9}$$

$$\mathrm{FAR}_c = \frac{\mathrm{FP}_c}{\mathrm{FP}_c + \mathrm{TN}_c} = 1 - \mathrm{Specificity}_c \tag{10}$$

Precision, recall, F1-score, specificity and FAR were macro-averaged over the 31 classes with Eq (11), so that every class contributes equally regardless of its size:

$$M_{\mathrm{macro}} = \frac{1}{31} \sum_{c=1}^{31} M_c \tag{11}$$

Per-class classification reports, confusion matrices, training time and the prediction time for the whole test partition were also recorded. Macro F1-score and macro recall were used as the primary ranking criteria, with accuracy, specificity, FAR and computational cost as supporting evidence. Because the FAR is macro-averaged over one-versus-rest problems, it should not be interpreted as the fraction of normal traffic flagged as malicious; this distinction is examined in the next section.

Use of generative AI

During the preparation of this manuscript, the authors used Claude (Anthropic) to help draft and edit the text from the PhD thesis and to prepare the figures from the recorded simulation outputs. The authors reviewed and edited all content and take full responsibility for the published article.

Results and discussion

Impact of the attacks on network behavior

Figs 5–7 show, for the baseline and for each of the thirty attacks, the average power of every reporting mote, split into LPM, CPU, radio-listen and radio-transmit power, and Table 8 lists the network averages of all scenarios. The per-mote values were read from the Cooja Collect View charts; their averages agree with Table 8 to within 2% in 28 of the 30 scenarios with a legible node table. Radio duty cycle is not plotted separately because, on the Sky platform, it is proportional to radio power: 1% of listen time corresponds to 0.60 mW and 1% of transmit time to 0.53 mW. Collect View lists only the motes whose reports reach the sink, so a missing mote (× in Figs 5–7) is itself evidence of disruption. The original Collect View charts and the network map of every scenario are provided in S1–S31 Figs. Because Cooja placed the motes at random positions in each scenario, per-mote values should be compared within a scenario rather than mote by mote across scenarios.

Fig 5. Average power of each reporting mote for the baseline and the ten perception-layer attacks. Power is split into LPM, CPU, radio-listen and radio-transmit power, read from the Cooja Collect View charts (originals in S1–S11 Figs). Each row has its own power scale; where the chart labels were truncated in the screenshots, motes 10–18 are assumed to be in ascending order. Gray band: attacker motes 19–21; ×: legitimate mote that did not report; dashed line: baseline mean, drawn where the scale allows; ratios in parentheses are relative to the baseline mean. † Estimated from the chart because the node table was not legible.

Baseline (S1 Fig). All twenty non-root motes reported to the sink and consumed 0.93–1.06 mW each, of which about 42% was CPU power and 40% radio listening; the radio was active for about 0.7% of the time. The network averages were a power of 0.995 mW, a listen duty cycle of 0.665% and a transmit duty cycle of 0.046%, and each mote delivered an average of 31.15 packets to the sink without loss during the capture.

Perception-layer attacks. Seven of the ten perception-layer attacks occupied the radio channel and dominated the energy budget of the legitimate motes (Fig 5), whereas energy exhaustion, MAC spoofing and eavesdropping left little or no footprint.

Constant jamming (S2 Fig). The attackers multicast 60-byte garbage frames every 50 ms. Only 8 of the 17 legitimate motes (2, 3, 4, 9, 12, 13, 15 and 17) still reached the sink. Each of them consumed 4.0–6.4 mW, about 80% of it for radio listening, because the continuous traffic kept their radios awake. The mean of 5.37 mW is 5.4 times the baseline, and only 1.4 packets per mote were delivered.

Deceptive jamming (S3 Fig). The attackers multicast legitimate-looking sensor readings at 16 packets/s. Sixteen motes reported (all except mote 5), and their power rose to 4.3–12.4 mW, again mostly because of radio listening. The node table of this scenario was not legible, but the chart gives a mean of about 9.2 mW, more than nine times the baseline.

Reactive jamming (S4 Fig). The attackers transmitted a noise burst only when they sensed channel activity. This produced the most uneven load of the experiment: motes 2, 3, 15 and 16 stayed below about 2 mW, whereas the other eleven reporting motes consumed 20–41 mW and kept their radios on for about 31–63% of the time. Motes 5 and 17 did not report, the mean reached 21.1 mW (21 times the baseline) and only 2.1 packets per mote were delivered.

Random jamming (S5 Fig). The attackers alternated random silent periods with bursts of 5–14 packets. All 17 motes reported, but their power varied widely, from about 1.4 mW (mote 12) to 19 mW (mote 8), with radio listening accounting for about 90% of the total. The mean was 10.1 mW, and deliveries fell to 3.5 packets per mote.

Energy exhaustion (S6 Fig). Each send event of the attackers was amplified into 50 protocol-conformant packets. The legitimate motes were unaffected: all 17 reported, each consumed 0.91–1.06 mW with the same CPU and listen shares as in the baseline, and the mean of 0.984 mW is practically the same as the baseline value. The extra cost falls on the attackers themselves, which do not appear in Collect View, so this attack must be detected from traffic rather than from the energy of the legitimate motes.

Collision (S7 Fig). The attackers sent short collision bursts every 50 ms. Sixteen motes reported (mote 14 did not), and each consumed 3.5–10.6 mW, mostly for radio listening, with a transmit share about twice

Table 8. Network-average statistics recorded for the baseline and each attack scenario (Collect View and PowerTracker).

Scenario	Motes	Received (packets)	Lost (packets)	CPU (mW)	LPM (mW)	Listen (mW)	Transmit (mW)	Total power (mW)	Listen duty cycle (%)	Transmit duty cycle (%)
Baseline (normal operation)	20	31.150	0.000	0.421	0.151	0.399	0.024	0.995	0.665	0.046
Perception layer										
Constant jamming	8	1.375	0.125	0.553	0.147	4.277	0.394	5.370	7.128	0.743
Deceptive jamming	16	n.l.	n.l.	n.l.	n.l.	n.l.	n.l.	n.l.	n.l.	n.l.
Reactive jamming	15	2.133	0.267	1.418	0.121	19.342	0.214	21.095†	32.236	0.403
Random jamming	17	3.471	0.118	0.828	0.138	8.909	0.241	10.116†	14.849	0.454
Energy exhaustion	17	34.294	0.000	0.416	0.151	0.395	0.022	0.984	0.658	0.041
Collision	16	1.500	0.000	0.750	0.141	6.826	0.475	8.192	11.377	0.895
MAC spoofing	17	8.941	0.176	0.433	0.150	0.725	0.204	1.512	1.209	0.384
De-synchronization	16	1.938	0.000	0.807	0.139	9.092	0.273	10.311†	15.153	0.514
Eavesdropping	17	23.000	0.000	0.422	0.151	0.399	0.025	0.997	0.664	0.048
Unfairness	16	1.563	0.000	1.567	0.116	19.128	0.284	21.095†	31.879	0.532
Network layer										
Sinkhole	17	23.706	0.000	0.413	0.151	0.396	0.020	0.979	0.659	0.037
Blackhole	17	27.353	0.059	0.398	0.151	0.399	0.030	0.979	0.665	0.057
Selective forwarding	17	17.118	0.000	0.427	0.151	0.407	0.037	1.021	0.679	0.069
Sybil	17	13.471	0.000	0.402	0.151	0.646	0.046	1.246	1.077	0.087
Wormhole	17	11.118	0.000	0.400	0.151	0.601	0.070	1.223	1.002	0.132
Hello flood	8	1.375	0.000	0.596	0.145	4.643	0.324	5.708	7.738	0.611
Decreased rank	17	25.765	0.000	0.419	0.151	0.541	0.038	1.149	0.902	0.071
Increased rank	17	16.235	0.000	0.438	0.150	0.574	0.024	1.186	0.956	0.046
Version number modification	17	3.294	0.294	1.062	0.131	2.611	1.592	5.396	4.351	2.998
Routing replay	17	2.824	0.176	0.836	0.138	1.861	1.584	4.419	3.101	2.982
Application layer										
Data falsification	17	34.000	0.000	0.413	0.151	0.396	0.025	0.985	0.660	0.047
Message forgery	17	18.294	0.000	0.425	0.151	0.415	0.038	1.029	0.692	0.072
Malicious code injection	17	26.647	0.000	0.409	0.151	0.402	0.028	0.991	0.670	0.054
Buffer overflow	17	16.529	0.059	0.404	0.151	0.421	0.047	1.023	0.701	0.088
Application-level DoS	16	2.438	2.563	0.431	0.150	0.675	0.077	1.333	1.125	0.144
FOTA hijacking	17	14.176	0.647	0.422	0.151	1.052	0.055	1.679	1.753	0.103
Brute force	17	4.176	2.941	0.445	0.150	0.620	0.071	1.286	1.033	0.134
Repudiation	17	23.647	0.000	0.403	0.151	0.405	0.036	0.995	0.675	0.068
Time synchronization	20‡	15.050	0.000	0.792	0.140	4.392	2.905	8.228	7.321	5.471
Data replay	17	23.882	0.000	0.394	0.152	0.399	0.033	0.977	0.665	0.062

Motes = number of reporting motes, i.e., motes whose reports reached the sink and appear in the Collect View node table; all other values are averages over these motes, taken from the last row of the node table for each scenario. Packet counts depend on the length of each capture and are indicative only. † Total power was truncated in the recorded screenshot and is given as the sum of its four components. ‡ The three attacker motes also reported to the sink in this scenario and are included in the averages. n.l. = not legible in the recorded screenshot.

that of the baseline, which is consistent with retransmissions. The mean of 8.19 mW is 8.2 times the baseline, and only 1.5 packets per mote reached the sink.

MAC spoofing (S8 Fig). The attackers changed their link-layer address every 2 s and announced each new identity with a hello message. Power rose only moderately, to 1.1–3.0 mW per mote, but the transmit share of the total increased from 2.5% to about 14%. Mote 17 was the most affected, with about 1.1 mW of transmit power, 38% of its total. All motes reported, and 0.18 packets per mote were lost.

De-synchronization (S9 Fig). The attackers multicast false MAC timestamps four times per second. Sixteen motes reported (mote 8 did not). Motes 2 and 3 stayed at about 2.6–3.4 mW, whereas the others consumed 6–16 mW, almost entirely for radio listening. The mean of 10.3 mW is ten times the baseline, and deliveries fell to 1.9 packets per mote.

Eavesdropping (S10 Fig). The attackers disabled duty cycling and listened continuously without transmitting. The legitimate motes showed no change: all 17 reported, each consumed 0.98–1.12 mW and the mean of 0.997 mW matches the baseline. Like energy exhaustion, this passive attack is invisible in the energy of the legitimate motes because its cost falls on the attackers.

Unfairness (S11 Fig). The attackers bypassed duty cycling and hogged the channel by sending 50 packets/s. Sixteen motes reported (mote 5 did not), and all of them were heavily loaded: power ranged from 7.2 mW (mote 3) to 35 mW (motes 4, 9 and 18), radio listening made up about 92% of the total and duty cycles ranged from about 11% to 55%. The mean of 21.1 mW equals that of reactive jamming, the highest of the experiment, and only 1.6 packets per mote were delivered.

Network-layer attacks. Seven of the ten network-layer attacks changed the mean power of the legitimate motes by no more than about 25% (Fig 6). In every recorded map, all motes lie inside the 50 m radio range of the root, so the legitimate motes could reach the sink directly. This may explain why attacks that lure traffic into poor routes had little effect on energy in this network. By contrast, hello flood and the two control-message attacks, version number modification and routing replay, left strong footprints.

Fig 6. Average power of each reporting mote for the ten network-layer attacks (originals in S12–S21 Figs). The baseline row is repeated for reference and the other details are as in Fig 5.

Sinkhole (S12 Fig). The attackers advertised an artificially low rank every 5 s to attract traffic. All 17 motes reported, and their power stayed at 0.93–1.06 mW, with the same breakdown as the baseline; only motes 5 and 6 showed a higher transmit share, about twice the baseline share. The mean of 0.979 mW is 2% below the baseline.

Blackhole (S13 Fig). The attackers advertised a low rank and then removed their preferred parent, so attracted packets were dropped. The energy profile of the legitimate motes was practically unchanged (0.92–1.06 mW per mote; mean 0.979 mW), but a few packets were lost (0.06 per mote), so this attack affected delivery rather than energy.

Selective forwarding (S14 Fig). The attackers switched randomly between forwarding and dropping. Most motes stayed at the baseline level, but the power of one mote rose to about 1.3 mW, with a transmit share of about 15%, and that of mote 9 to about 1.1 mW. The mean of 1.021 mW is only 3% above the baseline.

Sybil (S15 Fig). The attackers cloned the identity of mote 5 and sent routing packets every 2 s. All motes showed a moderate and uniform rise, to 1.1–1.5 mW, caused mainly by increased radio listening, and the mean of 1.246 mW is 25% above the baseline.

Wormhole (S16 Fig). The attackers advertised a low rank and multicast tunnel-encapsulated packets every 2 s. The power rise was uneven: motes 7, 12, 13 and 16 reached about 1.4–1.6 mW, whereas motes 2, 3 and 4 stayed at the baseline level. The mean was 1.223 mW, 23% above the baseline.

Hello flood (S17 Fig). The attackers broadcast routing-style hello messages at 25 packets/s. As under constant jamming, only eight motes (2, 5, 9, 10, 12, 14, 17 and 18) reached the sink. Each consumed 4.7–6.9 mW, mostly for radio listening, and only 1.4 packets per mote were delivered; the mean of 5.71 mW is 5.7 times the baseline.

Decreased rank (S18 Fig). The attackers forced their advertised rank down to 256 and reset the DIO timer to advertise it immediately. Power rose modestly and unevenly, to 0.91–1.39 mW per mote (mean 1.149 mW, 15% above the baseline).

Increased rank (S19 Fig). The attackers advertised an extremely high rank (10,000) and also reset the DIO timer. The effect was similar to that of decreased rank: 0.93–1.41 mW per mote, highest at motes 5, 10 and 6, and a mean of 1.186 mW (19% above the baseline).

Version number modification (S20 Fig). The attackers incremented the DODAG version every 10 s, forcing repeated global repairs. Every mote was affected almost equally (4.8–6.3 mW), and radio transmission rose to about 30% of the total power, compared with 2.5% in the baseline. The network-average transmit duty cycle increased about 65-fold, to 3.00%, the mean power was 5.40 mW (5.4 times the baseline) and 0.29 packets per mote were lost.

Routing replay (S21 Fig). The attackers re-advertised stale rank and version information every 6 s. The pattern resembled that of the version number modification attack: the power of all motes rose to 3.6–5.1 mW, the transmit share reached about 36% and the network transmit duty cycle rose to 2.98%. The mean was 4.42 mW (4.4 times the baseline), and 0.18 packets per mote were lost.

Application-layer attacks. The application-layer attacks kept the routing paths legitimate and altered the content, rate or meaning of application messages (Fig 7). Six of them were invisible in the energy of the legitimate motes, two were visible mainly as packet loss, and FOTA hijacking and time synchronization produced distinct energy patterns.

Fig 7. Average power of each reporting mote for the ten application-layer attacks (originals in S22–S31 Figs). The baseline row is repeated for reference and the other details are as in Fig 5. ‡ Includes the three attacker motes, which reported in this scenario.

Data falsification (S22 Fig). The attackers sent reports with implausible sensor values (for example, a temperature of 9999). All motes stayed at the baseline level (0.94–1.06 mW; mean 0.985 mW), because the attack changed only the content of packets sent at the normal rate.

Message forgery (S23 Fig). The attackers sent a fabricated critical alarm every 4 s. The legitimate motes were unaffected (0.98–1.12 mW; mean 1.029 mW, 3% above the baseline).

Malicious code injection (S24 Fig). The attackers sent executable-like payloads with a fake root command every 5 s. The energy profile was the same as in the baseline (0.94–1.05 mW; mean 0.991 mW).

Buffer overflow (S25 Fig). The attackers sent oversized payloads with padding every 3 s. The legitimate motes stayed at 0.96–1.12 mW (mean 1.023 mW), with only a slightly larger transmit share than in the baseline.

Application-level DoS (S26 Fig). The attackers flooded the sink with heavy payloads at about 20 packets/s each. The energy effect was moderate (1.1–1.9 mW; mean 1.333 mW, 1.3 times the baseline), and mote 5 did not report, but delivery collapsed: on average, 2.56 packets per mote were lost and only 2.44 were received, whereas no packets were lost in the baseline.

FOTA hijacking (S27 Fig). The attackers multicast a forged firmware block with an update command and a reboot instruction every 4 s. Power rose unevenly, mainly through radio listening, and ranged from about 0.9 mW (mote 8) to 3.4 mW (mote 13); the mean was 1.679 mW (1.7 times the baseline) and 0.65 packets per mote were lost.

Brute force (S28 Fig). The attackers sent about ten password guesses per second. Power rose only moderately (1.1–1.7 mW; mean 1.286 mW), but 2.94 packets per mote were lost, the highest loss of the experiment, and only 4.2 were received.

Repudiation (S29 Fig). The attackers sent commands without a sender identifier or audit fields every 5 s. No energy effect was visible (0.94–1.07 mW; mean 0.995 mW).

Time synchronization (S30 Fig). The attackers broadcast an application clock shifted by +999,999,999 s every 3 s. This is the only scenario in which the three attackers also appeared in Collect View, so twenty motes reported. Three motes, most likely the attackers, consumed about 48 mW each, with duty cycles of about 79%, whereas the other seventeen averaged about 1.3 mW. Hence, the network mean of 8.23 mW in Table 8 mainly reflects the attackers' own transmissions rather than a load imposed on the legitimate motes.

Data replay (S31 Fig). The attackers resent otherwise valid reports with a fixed, stale sequence number. The legitimate motes stayed at the baseline level (0.92–1.09 mW; mean 0.977 mW).

Summary of the attack footprints. Fig 8 compares the network averages of all scenarios on a common scale. Three groups emerge. Channel-occupying attacks (constant, deceptive, reactive and random

jamming, collision, de-synchronization, unfairness and hello flood) raised the mean power of the legitimate motes by a factor of 5–21 and reduced deliveries to 1.4–3.5 packets per mote. The two control-message attacks, version number modification and routing replay, raised radio transmission sharply on every mote. The remaining attacks raised the mean power of the legitimate motes by at most 70%; some of them were visible mainly as packet loss (application-level DoS, brute force, FOTA hijacking and, to a lesser extent, blackhole), whereas energy exhaustion, eavesdropping, sinkhole and the payload attacks left practically no energy trace. Time synchronization is a special case because its network average includes the attackers.

Fig 8. Average node power and radio-listen duty cycle for the baseline and each attack scenario. The vertical line marks the baseline value. ‡ The time-synchronization average includes the three attacker motes.

These footprints matter for detection. An attack with a strong energy or delivery signature is not necessarily easy to recognize from packet fields: reactive jamming has one of the strongest energy signatures, yet, as shown below, it is the hardest class for the packet-level detector. Conversely, attacks that leave no energy trace can still be recognized from their packets. Packet-level and energy-level evidence are therefore complementary, which motivates the combination of packet-level detection with cross-layer telemetry discussed later.

Overall detection performance

Table 9 reports the performance of all sixteen runs. RF + XGBoost (Run 4) outperformed every other architecture, with 85.91% accuracy, 86.80% macro precision, 82.61% macro recall, 84.52% macro F1-score, 99.49% macro specificity and a macro FAR of 0.51%. The stacked ET + LightGBM model (Run 10) ranked second, with 72.87% accuracy and 57.06% macro F1. The remaining thirteen architectures formed a tight cluster, with accuracy between 67.48% and 69.86%, macro recall between 46.14% and 49.78% and macro F1 between 47.75% and 52.55%. Runs 6 and 15 returned identical metrics, which confirms that the pipeline is deterministic under the recorded configuration.

Table 9. Overall performance of the sixteen experimental runs on the test partition of 542,529 records (%).

Run	Hybrid model	Accuracy	Macro precision	Macro recall	Macro F1-score	Macro specificity	Macro FAR
1	AdaBoost + CatBoost	68.99	70.45	48.86	51.10	98.88	1.12
2	CatBoost + MLP	68.42	70.45	47.01	49.31	98.84	1.16
3	ET + LightGBM	68.68	78.70	47.85	50.88	98.86	1.14
4	**RF + XGBoost**	**85.91**	**86.80**	**82.61**	**84.52**	**99.49**	**0.51**
5	HistGB + LightGBM	68.51	72.98	48.19	50.66	98.86	1.14
6	XGBoost + LightGBM	68.85	76.04	48.48	50.76	98.87	1.13
7	CNN + GRU	68.33	66.83	48.35	50.23	98.86	1.14
8	CNN + LSTM	68.44	67.47	48.18	50.10	98.86	1.14
9	CNN + SimpleRNN	67.48	60.83	46.14	47.75	98.81	1.19
10	ET + LightGBM (stacking)	72.87	77.13	53.47	57.06	99.01	0.99
11	ET + MLP	68.64	72.23	47.75	50.68	98.85	1.15
12	LightGBM + MLP	68.45	80.46	48.50	51.01	98.86	1.14
13	XGBoost + MLP	69.86	77.16	49.78	52.55	98.91	1.09
14	RF + MLP	69.07	72.05	48.76	51.50	98.87	1.13
15	XGBoost + LightGBM (repeat)	68.85	76.04	48.48	50.76	98.87	1.13
16	RF + LightGBM	68.81	77.96	48.38	50.80	98.87	1.13

Best values are shown in bold. Run 15 repeats Run 6 and is excluded from the ranking of unique architectures.

The gap between accuracy and macro F1 separates the models more sharply than accuracy alone (Fig 9).

For the thirteen clustered models, macro F1 was on average 18.1 percentage points lower than accuracy, showing that they classified the large classes well but failed on many minority classes. For RF + XGBoost, the gap was only 1.39 points, and its macro F1 exceeded that of the second-ranked model by 27.46 points. The best model also combined the highest specificity with the lowest FAR (Fig 10), so its gain in recall was not obtained by assigning attack labels indiscriminately.

Fig 9. Accuracy and macro F1-score of the fifteen unique hybrid architectures. A long connector indicates that high accuracy hides poor minority-class performance.

Fig 10. Macro F1-score versus macro false alarm rate of the fifteen unique architectures. The upper-left region is better.

Class-level behavior of the selected model

Table 10 lists the per-class precision, recall and F1-score of RF + XGBoost on the test partition. No class had zero recall. Twelve classes reached an F1-score of at least 95%: constant, deceptive and random jamming, collision, unfairness, de-synchronization, MAC spoofing, hello flood, Sybil, wormhole, FOTA hijacking and time synchronization. Four RPL control-plane attacks (decreased rank, increased rank, version number modification and routing replay) scored between 92.39% and 94.84%. Averaged by layer, the F1-score was highest for network-layer attacks (91.08%), followed by perception-layer (87.03%) and application-layer attacks (76.66%).

Table 10. Per-class precision (P), recall (R) and F1-score (F1) of RF + XGBoost on the test partition (%).

Perception layer	P	R	F1	Network layer	P	R	F1	Application layer	P	R	F1
Constant jamming	99.42	99.46	99.44	Sinkhole	89.84	83.71	86.67	Data falsification	86.85	80.21	83.39
Deceptive jamming	99.54	99.53	99.53	Blackhole	82.23	70.11	75.69	Message forgery	74.39	57.52	64.88
Reactive jamming	20.99	20.22	20.60	Selective forwarding	83.54	72.06	77.38	Malicious code injection	72.96	62.35	67.24
Random jamming	97.71	97.82	97.76	Sybil	99.02	98.13	98.58	Buffer overflow	70.06	60.10	64.70
Energy exhaustion	86.66	70.33	77.64	Wormhole	98.47	97.54	98.00	Application-level DoS	73.84	79.69	76.65
Collision	99.56	99.50	99.53	Hello flood	99.54	99.38	99.46	FOTA hijacking	98.55	96.94	97.74
MAC spoofing	98.61	97.77	98.19	Decreased rank	94.20	90.65	92.39	Brute force	69.54	74.39	71.89
De-synchronization	99.48	99.34	99.41	Increased rank	94.52	91.76	93.12	Repudiation	72.65	59.36	65.34
Eavesdropping	85.41	72.74	78.56	Version number modification	94.89	94.39	94.64	Time synchronization	98.01	96.17	97.08
Unfairness	99.67	99.53	99.60	Routing replay	95.26	94.42	94.84	Data replay	84.16	72.08	77.65
Layer mean	**88.70**	**85.62**	**87.03**	**Layer mean**	**93.15**	**89.22**	**91.08**	**Layer mean**	**80.10**	**73.88**	**76.66**

Normal traffic: P 71.08, R 73.85, F1 72.44. Macro average over all 31 classes: P 86.80, R 82.61, F1 84.52.

Five classes remained below 70% F1: reactive jamming (20.60%), buffer overflow (64.70%), message forgery (64.88%), repudiation (65.34%) and malicious code injection (67.24%). The confusion matrix in Fig 11 shows why. The dominant error is a two-way confusion between Normal traffic and reactive jamming: because the reactive jammer transmits only when legitimate traffic is present, its packets coincide in time and structure with normal packets, and the six packet-level features cannot separate them reliably. Normal traffic is also occasionally confused with application-level DoS and brute-force records. The remaining application-layer errors are concentrated inside the application block, where code injection, buffer overflow, repudiation and message forgery are partly confused with brute-force and application-level DoS traffic, all of which are attacker-to-sink transfers with similar addressing and protocol fields. Apart from the

Normal–reactive-jamming confusion, errors between layers are comparatively rare, which indicates that the layer of an attack is easier to identify than its exact type.

Fig 11. Confusion matrix of RF + XGBoost on the test partition, with classes grouped by layer. N = Normal; the columns follow the same order as the rows.

The per-class report also clarifies how the macro FAR should be interpreted. The recall of the Normal class was 73.85% (precision 71.08%), so 26.15% of the normal test records were assigned to an attack class, mostly reactive jamming. In a binary deployment in which every non-normal prediction raises an alarm, the alarm rate on normal traffic would be about 26%, far higher than the 0.51% macro FAR. The macro FAR stays low because the negative pool of each one-versus-rest problem contains the records of thirty other classes, which inflates TN_c; the same effect keeps macro specificity near 99% for every model even when macro recall is below 50%. Accordingly, high specificity should always be read together with macro recall and class-level results.

Effect of the fusion strategy and model family

Runs 3 and 10 provide the most controlled comparison in the experiment because they use the same ET and LightGBM families and differ only in how the base predictions are combined. Stacking with a logistic-regression meta-learner raised accuracy from 68.68% to 72.87%, macro recall from 47.85% to 53.47% and macro F1 from 50.88% to 57.06% (+6.18 points), and it lowered the macro FAR from 1.14% to 0.99%. The weakest class also improved, from 1.30% recall for eavesdropping with averaging to 4.82% with stacking, and the stacked confusion matrix showed stronger diagonal coverage across more classes. Thus, how base models are combined can matter as much as which models are combined, because equal averaging of two learners that make similar errors simply reproduces those errors.

None of the five hybrids that paired a tree ensemble with an MLP approached RF + XGBoost, and four of them required the longest training times of the experiment (24.4–31.5 min). The three convolutional–recurrent hybrids performed almost identically (67.48–68.44% accuracy and 47.75–50.23% macro F1): CNN + LSTM achieved the highest accuracy, CNN + GRU the highest macro recall and macro F1, and CNN + SimpleRNN trained fastest but left seven classes with zero recall. Within the six-feature representation and training budget used here, deeper architectures did not translate into better detection. This is a statement about the present experiment rather than a general ranking of tree-based and deep models.

Table 11 summarizes the class-level recall pattern of the fifteen unique architectures. Deceptive jamming, collision, unfairness, de-synchronization and Sybil consistently appeared among the best-detected classes, with reported recalls above 92%, whereas eavesdropping, selective forwarding, energy exhaustion and data replay were the weakest classes for most models. Eight architectures left between two and seven classes with zero recall. RF + XGBoost is the only model whose weakest class still exceeded 20% recall; for every other model, the weakest class stayed below 5%.

The configurations in Table 7 suggest one reason for the large margin of RF + XGBoost. Its RF component was grown without a depth limit, whereas the RF models of Runs 14 and 16 and all ET models were limited to a depth of 12 and the boosting models to a depth of 6. Fully grown trees can isolate rare combinations of addresses, protocol values and Info strings that shallower trees merge, which is consistent with the much higher recall of RF + XGBoost on minority classes such as eavesdropping (72.74%), energy exhaustion (70.33%) and selective forwarding (72.06%), for which the other models mostly stayed below 8%. Consequently, the comparison is not a pure test of the RF + XGBoost pairing, because tree depth and fusion partner are confounded; an ablation of single models and depth settings is needed to separate their effects.

Computational cost

Fig 12 compares the training and prediction times recorded in the Google Colaboratory notebook session. RF + XGBoost required 16.87 min of training and 82.13 s to classify all 542,529 test records, which corresponds to about 6,606 predictions per second. It was neither the fastest nor the slowest model: the CNN–recurrent

Table 11. Class-level recall patterns of the fifteen unique architectures (%).

Run	Hybrid model	Three highest-recall classes	Three lowest-recall classes	Zero-recall classes
1	AdaBoost + CatBoost	Deceptive jamming 93.90; unfairness 92.50; de-synchronization 92.33	Eavesdropping 0.00; data replay 0.00; energy exhaustion 0.00	4
2	CatBoost + MLP	Deceptive jamming 92.54; unfairness 92.50; Sybil 92.10	Eavesdropping 0.00; data replay 0.00; energy exhaustion 0.00	4
3	ET + LightGBM	Deceptive jamming 95.25; collision 94.80; unfairness 94.08	Eavesdropping 1.30; energy exhaustion 1.53; selective forwarding 1.53	0
4	RF + XGBoost	Deceptive jamming 99.53; unfairness 99.53; collision 99.50	Reactive jamming 20.22; message forgery 57.52; repudiation 59.36	0
5	HistGB + LightGBM	Deceptive jamming 95.44; collision 94.72; unfairness 94.38	Eavesdropping 1.40; selective forwarding 2.41; data falsification 3.03	0
6	XGBoost + LightGBM	Deceptive jamming 95.50; collision 94.86; unfairness 94.41	Eavesdropping 1.30; energy exhaustion 1.53; selective forwarding 2.00	0
7	CNN + GRU	Deceptive jamming 93.97; collision 93.13; unfairness 92.50	Blackhole 0.00; eavesdropping 0.00; energy exhaustion 0.00	5
8	CNN + LSTM	Deceptive jamming 94.54; de-synchronization 93.45; collision 92.87	Blackhole 0.00; eavesdropping 0.00; energy exhaustion 0.00	5
9	CNN + SimpleRNN	Deceptive jamming 93.25; unfairness 92.50; Sybil 92.14	Blackhole 0.00; eavesdropping 0.00; message forgery 0.00	7
10	ET + LightGBM (stacking)	Deceptive jamming 96.00; collision 95.58; unfairness 95.34	Eavesdropping 4.82; selective forwarding 6.68; energy exhaustion 7.37	0
11	ET + MLP	Unfairness 92.50; deceptive jamming 92.45; Sybil 92.14	Eavesdropping 0.00; data replay 0.00; energy exhaustion 0.00	4
12	LightGBM + MLP	Deceptive jamming 95.36; collision 94.24; unfairness 93.93	Energy exhaustion 1.29; eavesdropping 1.30; selective forwarding 1.48	0
13	XGBoost + MLP	Deceptive jamming 95.02; collision 93.47; de-synchronization 92.93	Eavesdropping 0.00; energy exhaustion 0.00; selective forwarding 0.46	2
14	RF + MLP	Deceptive jamming 93.76; unfairness 92.50; Sybil 92.14	Eavesdropping 0.00; data replay 0.00; energy exhaustion 0.00	4
16	RF + LightGBM	Deceptive jamming 95.40; collision 94.92; unfairness 94.42	Energy exhaustion 1.29; eavesdropping 1.30; selective forwarding 1.62	0

Run 15 repeats Run 6 and produced identical class-level results.

hybrids trained in 3.3–6.1 min and predicted in 8.7–20.9 s, whereas the LightGBM-based hybrids, including the stacked model, needed 181–252 s for prediction. Given its 27-point lead in macro F1, RF + XGBoost offers the best performance–cost balance in this experiment. These timings are observations of one session rather than hardware-independent benchmarks, and inference on constrained motes was not measured.

Fig 12. Observed training time and prediction time of the sixteen runs. RF + XGBoost is highlighted.

Comparison with related studies

Table 12 places the present results next to related Cooja- and Contiki-based studies. Their reported accuracies of about 92–99.8% are higher than the 85.91% obtained here, but the tasks are not comparable. Those studies distinguish two to seven classes, mostly of routing attacks, and often use a separate model for each attack scenario, whereas this study trains a single detector on thirty attacks from three layers plus normal traffic. A 31-class detector must also separate attack types that share packet-level signatures, such as the attacker-to-sink application payloads, a difficulty that binary normal-versus-attack models do not face. A fair comparison would require identical attack sets, class counts, preprocessing and validation protocols. The contribution of this work therefore lies in the breadth of the multiclass problem and the transparency of the class-level evaluation rather than in a higher headline accuracy.

Table 12. Comparison with related Cooja- and Contiki-based intrusion detection studies.

Study	Data source	Attacks and classes	Layers covered	Best model	Reported result
Kamel and Elhamayed [30]	Cooja (IoT healthcare)	Five routing attacks, each against normal traffic	Network	CNN	Accuracy 90.40–98.57%
Liu et al. [20]	Contiki-NG with NSL-KDD attack patterns	Seven classes (flooding, DoS, scanning)	Network/transport	XGBoost	Accuracy 97.0%
Choudhary et al. [23]	Cooja (Contiki OS 3.0)	Three attacks, each against normal traffic	Network	CNN with Aquila optimizer	Accuracy 99.22–99.77%
Farea and Küçük [24]	Cooja (KoU-6LoWPAN-IoT)	Three attacks + normal (4 classes)	Network	ANN	Accuracy 99.84%
Soumya et al. [25]	Cooja (Contiki-NG)	Blackhole against normal traffic	Network	Autoencoder, Isolation Forest	Accuracy about 92%
Panda and Muthuraman [26]	Cooja (Contiki-NG)	Four attacks + benign (5 classes)	Network	Soft-voting ensemble	Accuracy 99.47%
This study	**Cooja (Contiki OS 2.7)**	**Thirty attacks + normal (31 classes)**	**Perception, network, application**	**RF + XGBoost**	**Accuracy 85.91%; macro F1 84.52%**

Integration into the layered security framework

The experimental results were used to instantiate the detection core of the proposed layered security framework for IoT-enabled WSNs (Fig 13). From left to right in Fig 13, the framework collects traffic from the IoT-enabled WSN, preprocesses it with the same pipeline as in the experiment (missing-value encoding, training-set scaling and categorical encoding), organizes monitoring around perception-, network- and application-layer attacks and passes the records to the hybrid IDS engine. RF + XGBoost was selected for this engine on the basis of accuracy, macro recall, macro F1, specificity, FAR, class coverage and prediction cost, and it assigns each record to one of the 31 traffic classes through Eqs (2) and (3).

Fig 13. Proposed layered security framework for IoT-enabled WSNs. The shaded blocks were implemented and evaluated in this study.

A security decision engine then converts the predicted class into an action. As a framework-level rule, the risk of a record can be scored as $R(x) = \lambda_1 \,\mathrm{Sev}(\hat{y}) + \lambda_2 \,\mathrm{Conf}(x) + \lambda_3 \,\mathrm{Imp}(\hat{y})$, where Sev is the severity assigned to the predicted class, Conf the prediction confidence, Imp the expected impact of the threat and λ_1–λ_3 design weights. Normal traffic leads to continued monitoring; suspicious or low-confidence events are logged and raised as alerts (detect); confirmed moderate threats are contained, for example by blocking a source, isolating a node or re-selecting a route (detect and respond); and confirmed high-impact threats additionally trigger mitigation such as filtering, bypass routing or recovery (detect, respond and mitigate). The resilience mechanism aims to keep the network operating, or to restore it, during and after an attack.

The framework deliberately distinguishes what was measured from what is proposed. Traffic collection, preprocessing, the hybrid IDS engine, model evaluation, best-model selection and 31-class threat classification were implemented and evaluated in this study. The layer-wise monitoring branches, the decision engine, the response actions and the resilience mechanism are architectural components whose thresholds and weights remain to be defined and whose effectiveness, for example the time to detect, contain and recover and the packet delivery during an attack, remains to be measured.

Limitations

These findings are subject to several limitations. First, all evidence comes from Cooja simulation; some attacks are behavioral emulations of physical or software exploits, and real radios, interference, hardware diversity and mobility were not represented. Second, the models were compared on a single stratified 70:30 hold-out split without repeated seeds, cross-validation or confidence intervals, so the ranking reflects the observed experiment rather than an established statistical superiority; in addition, the MLP components of Runs 11–14 stopped at the 50-iteration limit before converging, so these hybrids may understate what a fully trained MLP could achieve. Third, the split was made at record level: packets from the same capture, including records that are close in time, appear in both partitions, and features such as Time and exact addresses may allow models, especially fully grown trees, to learn capture-specific patterns. Performance on captures from unseen topologies, seeds or attacker positions may therefore be lower, and scenario-disjoint validation is required. Fourth, the node layout was generated separately for each scenario, so part of the difference between scenarios may reflect topology rather than the attack alone. Moreover, the recorded network maps place every mote within the transmission range of the root, i.e., one radio hop from the sink, which limits the effect of routing attacks. Fifth, only six packet-level features were used; the power, duty-cycle and routing evidence recorded for every scenario was not part of the learning matrix, which probably explains the difficulty with reactive jamming. Sixth, the simulations used Contiki OS 2.7 rather than the actively maintained Contiki-NG, whose RPL and MAC implementations differ, and the per-mote power values in Figs 5–7 were read from the Collect View charts rather than from exported energy logs, so they are approximate where the chart labels were truncated or illegible. Finally, the decision, response and resilience components of the framework were not evaluated experimentally.

Conclusion

This study developed and evaluated a cross-layer approach to intrusion detection for IoT-enabled WSNs. Thirty attacks, ten at each of the perception, network and application layers, were implemented as executable Contiki OS 2.7 firmware in a 21-mote Cooja RPL network, and their packet captures, together with a normal baseline, formed a 31-class dataset of 1,808,427 records. The simulation evidence showed that the attacks leave distinct footprints: channel-occupying perception attacks raised the average power of the legitimate motes up to about 21 times the baseline value, control-plane attacks multiplied transmit duty cycles, and many application-layer attacks were almost invisible in energy terms.

Sixteen experimental runs representing fifteen hybrid architectures were compared on identical stratified partitions. RF + XGBoost achieved the best overall balance, with 85.91% accuracy, 82.61% macro recall, 84.52% macro F1-score, 99.49% macro specificity and a macro FAR of 0.51%; it detected every class at least partly and classified the full test partition in 82.13 s. Stacking improved the macro F1-score of an ET + LightGBM pair by 6.18 points over equal averaging of the same learners, whereas MLP and

convolutional–recurrent hybrids did not improve on tree-based designs. The class-level analysis showed that reactive jamming and several attacker-to-sink application payloads remain difficult to separate from normal and neighboring traffic with packet features alone, and that macro specificity and FAR must be interpreted together with macro recall.

The selected detector was integrated as the analytical core of a layered security framework that connects traffic collection, cross-layer monitoring, threat classification, decision-making, response, mitigation and resilience. Within the limits of a simulation study and a single hold-out split, the results show that broad multiclass detection across the IoT stack is achievable and provides a defensible foundation for resilient IoT-WSN security.

Future work

Future work will replicate the attacks in Contiki-NG with per-mote energy logged directly by the simulator, validate the framework on a physical or hardware-in-the-loop IoT-WSN testbed and measure packet delivery, latency, memory and energy overhead. The dataset will be extended with a fixed common layout for controlled comparisons, multi-hop topologies and more seeds, node densities, traffic loads and attacker positions, and it will be evaluated with repeated, scenario-disjoint cross-validation and confidence intervals. The feature set will be enriched with cross-layer telemetry such as RPL rank and parent changes, ETX, retransmissions, duty cycle and residual energy, processed hierarchically so that motes report lightweight indicators and gateways perform the heavier correlation. Open-set, incremental and confidence-based rejection mechanisms will be added for previously unseen attacks [50]. Finally, the security decision engine and the response policies, including the consequences of false positives, will be implemented, resilience will be measured directly through the time to detect, contain and recover, and the detector will be optimized for gateway- or edge-level deployment through model compression and feature reduction.

Supporting information

S1 Fig. Baseline (normal operation) scenario. Cooja network map and Collect View charts of average power and radio duty cycle per mote.

S2 Fig. Constant jamming scenario. Cooja network map and Collect View charts of average power and radio duty cycle per mote.

S3 Fig. Deceptive jamming scenario. Cooja network map and Collect View charts of average power and radio duty cycle per mote.

S4 Fig. Reactive jamming scenario. Cooja network map and Collect View charts of average power and radio duty cycle per mote.

S5 Fig. Random jamming scenario. Cooja network map and Collect View charts of average power and radio duty cycle per mote.

S6 Fig. Energy exhaustion scenario. Cooja network map and Collect View charts of average power and radio duty cycle per mote.

S7 Fig. Collision scenario. Cooja network map and Collect View charts of average power and radio duty cycle per mote.

S8 Fig. MAC spoofing scenario. Cooja network map and Collect View charts of average power and radio duty cycle per mote.

S9 Fig. De-synchronization scenario. Cooja network map and Collect View charts of average power and radio duty cycle per mote.

S10 Fig. Eavesdropping scenario. Cooja network map and Collect View charts of average power and radio duty cycle per mote.

S11 Fig. Unfairness scenario. Cooja network map and Collect View charts of average power and radio duty cycle per mote.

S12 Fig. Sinkhole scenario. Cooja network map and Collect View charts of average power and radio duty cycle per mote.

S13 Fig. Blackhole scenario. Cooja network map and Collect View charts of average power and radio duty cycle per mote.

S14 Fig. Selective forwarding scenario. Cooja network map and Collect View charts of average power and radio duty cycle per mote.

S15 Fig. Sybil scenario. Cooja network map and Collect View charts of average power and radio duty cycle per mote.

S16 Fig. Wormhole scenario. Cooja network map and Collect View charts of average power and radio duty cycle per mote.

S17 Fig. Hello flood scenario. Cooja network map and Collect View charts of average power and radio duty cycle per mote.

S18 Fig. Decreased rank scenario. Cooja network map and Collect View charts of average power and radio duty cycle per mote.

S19 Fig. Increased rank scenario. Cooja network map and Collect View charts of average power and radio duty cycle per mote.

S20 Fig. Version number modification scenario. Cooja network map and Collect View charts of average power and radio duty cycle per mote.

S21 Fig. Routing replay scenario. Cooja network map and Collect View charts of average power and radio duty cycle per mote.

S22 Fig. Data falsification scenario. Cooja network map and Collect View charts of average power and radio duty cycle per mote.

S23 Fig. Message forgery scenario. Cooja network map and Collect View charts of average power and radio duty cycle per mote.

S24 Fig. Malicious code injection scenario. Cooja network map and Collect View charts of average power and radio duty cycle per mote.

S25 Fig. Buffer overflow scenario. Cooja network map and Collect View charts of average power and radio duty cycle per mote.

S26 Fig. Application-level DoS scenario. Cooja network map and Collect View charts of average power and radio duty cycle per mote.

S27 Fig. FOTA hijacking scenario. Cooja network map and Collect View charts of average power and radio duty cycle per mote.

S28 Fig. Brute force scenario. Cooja network map and Collect View charts of average power and radio duty cycle per mote.

S29 Fig. Repudiation scenario. Cooja network map and Collect View charts of average power and radio duty cycle per mote.

S30 Fig. Time synchronization scenario. Cooja network map and Collect View charts of average power and radio duty cycle per mote.

S31 Fig. Data replay scenario. Cooja network map and Collect View charts of average power and radio duty cycle per mote.

Acknowledgments

The authors thank Dr. Khalid Mehmood and Dr. Muhammad Javed, members of the supervisory committee at the Gomal Research Institute of Computing, Gomal University, Dera Ismail Khan, for their guidance and support throughout this research.

References

1. Obaidat MA, Obeidat SA, Holst J, Al Hayajneh A, Brown J. A comprehensive and systematic survey on the Internet of Things: security and privacy challenges, security frameworks, enabling technologies, threats, vulnerabilities and countermeasures. Computers. 2020;9(2):44. doi: 10.3390/computers9020044
2. Nguyen DT, Trinh ML, Nguyen MT, Vu TC, Nguyen TV, Dinh LQ, et al. Security issues in IoT-based wireless sensor networks: classifications and solutions. Future Internet. 2025;17(8):350. doi: 10.3390/fi17080350
3. Winter T, Thubert P, Brandt A, Hui J, Kelsey R, Levis P, et al. RPL: IPv6 Routing Protocol for Low-Power and Lossy Networks. RFC 6550. Internet Engineering Task Force; 2012. doi: 10.17487/RFC6550
4. Mishra S, Albarakati A, Sharma SK. Cyber threat intelligence for IoT using machine learning. Processes. 2022;10(12):2673. doi: 10.3390/pr10122673
5. Kaur K, Kaur A, Gulzar Y, Gandhi V. Unveiling the core of IoT: comprehensive review on data security challenges and mitigation strategies. Front Comput Sci. 2024;6:1420680. doi: 10.3389/fcomp.2024.1420680
6. Sharma G, Vidalis S, Anand N, Menon C, Kumar S. A survey on layer-wise security attacks in IoT: attacks, countermeasures, and open-issues. Electronics. 2021;10(19):2365. doi: 10.3390/electronics10192365
7. Lim KS, Ooi SY, Sayeed MS, Chew YJ, Ahmad NM. Securing the Internet of Things: systematic insights into architectures, threats, and defenses. Electronics. 2025;14(20):3972. doi: 10.3390/electronics14203972

8. Madani P, Vlajic N. RSSI-based MAC-layer spoofing detection: deep learning approach. J Cybersecur Priv. 2021;1(3):453–469. doi: 10.3390/jcp1030023

9. Herrera-Loera C, Del-Valle-Soto C, Valdivia LJ, Vázquez-Castillo J, Mex-Perera C. A systematic review and energy-centric taxonomy of jamming attacks and countermeasures in wireless sensor networks. Sensors. 2026;26(2):579. doi: 10.3390/s26020579

10. Karlof C, Wagner D. Secure routing in wireless sensor networks: attacks and countermeasures. Ad Hoc Netw. 2003;1(2–3):293–315. doi: 10.1016/S1570-8705(03)00008-8

11. Tsao T, Alexander R, Dohler M, Daza V, Lozano A, Richardson M. A security threat analysis for the Routing Protocol for Low-Power and Lossy Networks (RPLs). RFC 7416. Internet Engineering Task Force; 2015. doi: 10.17487/RFC7416

12. Alfriehat N, Anbar M, Aladaileh M, Hasbullah I, Shurbaji TA, Karuppayah S, et al. RPL-based attack detection approaches in IoT networks: review and taxonomy. Artif Intell Rev. 2024;57(9):248. doi: 10.1007/s10462-024-10907-y

13. Tsaur W-J, Chang J-C, Chen C-L. A highly secure IoT firmware update mechanism using blockchain. Sensors. 2022;22(2):530. doi: 10.3390/s22020530

14. Almeghlef SM, AL-Malaise AL-Ghamdi A, Ramzan MS, Ragab M. Application layer-based denial-of-service attacks detection against IoT-CoAP. Electronics. 2023;12(12):2563. doi: 10.3390/electronics12122563

15. Hu J, Yang X, Yang L-X. A framework for detecting false data injection attacks in large-scale wireless sensor networks. Sensors. 2024;24(5):1643. doi: 10.3390/s24051643

16. Mustafa R, Sarkar NI, Mohaghegh M, Pervez S. A cross-layer secure and energy-efficient framework for the Internet of Things: a comprehensive survey. Sensors. 2024;24(22):7209. doi: 10.3390/s24227209

17. Ismail S, Dawoud DW, Reza H. Securing wireless sensor networks using machine learning and blockchain: a review. Future Internet. 2023;15(6):200. doi: 10.3390/fi15060200

18. Tavallaee M, Bagheri E, Lu W, Ghorbani AA. A detailed analysis of the KDD CUP 99 data set. In: 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications; 2009. pp. 1–6. doi: 10.1109/CISDA.2009.5356528

19. Sharafaldin I, Lashkari AH, Ghorbani AA. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In: Proceedings of the 4th International Conference on Information Systems Security and Privacy; 2018. pp. 108–116. doi: 10.5220/0006639801080116

20. Liu J, Kantarci B, Adams C. Machine learning-driven intrusion detection for Contiki-NG-based IoT networks exposed to NSL-KDD dataset. In: Proceedings of the 2nd ACM Workshop on Wireless Security and Machine Learning; 2020. pp. 25–30. doi: 10.1145/3395352.3402621

21. Dunkels A, Grönvall B, Voigt T. Contiki: a lightweight and flexible operating system for tiny networked sensors. In: Proceedings of the 29th Annual IEEE International Conference on Local Computer Networks; 2004. pp. 455–462. doi: 10.1109/LCN.2004.38

22. Österlind F, Dunkels A, Eriksson J, Finne N, Voigt T. Cross-level sensor network simulation with COOJA. In: Proceedings of the 31st IEEE Conference on Local Computer Networks; 2006. pp. 641–648. doi: 10.1109/LCN.2006.322172

23. Choudhary V, Tanwar S, Choudhury T. Generating IoT specific anomaly datasets using Cooja simulator (Contiki-OS) and performance evaluation of deep learning model coupled with Aquila optimizer. J Comput Sci. 2024;20(4):365–378. doi: 10.3844/jcssp.2024.365.378

24. Farea AH, Küçük K. Machine learning-based intrusion detection technique for IoT: simulation with Cooja. Int J Comput Netw Inf Secur. 2024;16(1):1–23. doi: 10.5815/ijcnis.2024.01.01

25. Soumya K, Deepika B, Akhila B, Revathi B, Vinaya CN, Pallam Setti S. Prediction and detection of black hole attack using the COOJA simulator. Int Res J Eng Technol. 2025;12(3):788–793.

26. Panda N, Muthuraman S. A multiclass machine learning framework for detecting routing attacks in RPL-based IoT networks using a novel simulation-driven dataset. Future Internet. 2026;18(1):35. doi: 10.3390/fi18010035

27. Wang A, Mohaisen A, Chen S. XLF: a cross-layer framework to secure the Internet of Things (IoT). In: 2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS); 2019. pp. 1830–1839. doi: 10.1109/ICDCS.2019.00181

28. Kore A, Patil S. Robust cross-layer security framework for Internet of Things enabled wireless sensor networks. In: 2020 International Conference on Emerging Smart Computing and Informatics (ESCI); 2020. pp. 142–147. doi: 10.1109/ESCI48226.2020.9167555

29. Upadhiyay A, Jain A. Cyber resilient framework with energy efficient swarm routing and ensemble threat detection in fog assisted wireless sensor networks. Sci Rep. 2025;15(1):36461. doi: 10.1038/s41598-025-21368-w

30. Kamel SOM, Elhamayed SA. Mitigating the impact of IoT routing attacks on power consumption in IoT healthcare environment using convolutional neural network. Int J Comput Netw Inf Secur. 2020;12(4):11–29. doi: 10.5815/ijcnis.2020.04.02

31. Wan Y, Xu K, Xue G, Wang F. IoTArgos: a multi-layer security monitoring system for Internet-of-Things in smart homes. In: IEEE INFOCOM 2020—IEEE Conference on Computer Communications; 2020. pp. 874–883. doi: 10.1109/INFOCOM41043.2020.9155424

32. Momand MD, Mohsin MK, Ihsanulhaq. Machine learning-based multiple attack detection in RPL over IoT. In: 2021 International Conference on Computer Communication and Informatics (ICCCI); 2021. pp. 1–8. doi: 10.1109/ICCCI50826.2021.9402388

33. Ullah I, Ullah A, Sajjad M. Toward a hybrid deep learning model for anomalous activities detection in Internet of Things networks. IoT. 2021;2(3):22. doi: 10.3390/iot2030022

34. Rabhi S, Abbes T, Zarai F. IoT routing attacks detection using machine learning algorithms. Wirel Pers Commun. 2022;128(3):1839–1857. doi: 10.1007/s11277-022-10022-7

35. Khan S, Khan MA, Alnazzawi N. Artificial neural network-based mechanism to detect security threats in wireless sensor networks. Sensors. 2024;24(5):1641. doi: 10.3390/s24051641

36. Vora V, Sinha R, Palan J, Kulkarni V. Machine learning-based intrusion detection system for IoT using Contiki-OS. In: 2024 3rd International Conference on Automation, Computing and Renewable Systems (ICACRS); 2024. pp. 542–548. doi: 10.1109/ICACRS62842.2024.10841743

37. Srivastava A, Jain U. Securing the future of IoT: a comprehensive framework for real-time attack detection and mitigation in IoT networks. In: 2023 14th International Conference on Computing Communication and Networking Technologies (ICCCNT); 2023. pp. 1–6. doi: 10.1109/ICCCNT56998.2023.10307306

38. Gnawali O, Levis P. The Minimum Rank with Hysteresis Objective Function. RFC 6719. Internet Engineering Task Force; 2012. doi: 10.17487/RFC6719

39. Finne N, Eriksson J, Voigt T, Suciu G, Sachian M-A, Ko J, et al. Multi-Trace: multi-level data trace generation with the Cooja simulator. In: 2021 17th International Conference on Distributed Computing in Sensor Systems (DCOSS); 2021. pp. 390–395. doi: 10.1109/DCOSS52077.2021.00068

40. Breiman L. Random forests. Mach Learn. 2001;45(1):5–32. doi: 10.1023/A:1010933404324

41. Geurts P, Ernst D, Wehenkel L. Extremely randomized trees. Mach Learn. 2006;63(1):3–42. doi: 10.1007/s10994-006-6226-1

42. Chen T, Guestrin C. XGBoost: a scalable tree boosting system. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining; 2016. pp. 785–794. doi: 10.1145/2939672.2939785

43. Ke G, Meng Q, Finley T, Wang T, Chen W, Ma W, et al. LightGBM: a highly efficient gradient boosting decision tree. Adv Neural Inf Process Syst. 2017;30:3146–3154.

44. Prokhorenkova L, Gusev G, Vorobev A, Dorogush AV, Gulin A. CatBoost: unbiased boosting with categorical features. Adv Neural Inf Process Syst. 2018;31:6638–6648.

45. Freund Y, Schapire RE. A decision-theoretic generalization of on-line learning and an application to boosting. J Comput Syst Sci. 1997;55(1):119–139. doi: 10.1006/jcss.1997.1504

46. Wolpert DH. Stacked generalization. Neural Netw. 1992;5(2):241–259. doi: 10.1016/S0893-6080(05)80023-1

47. Cho K, van Merriënboer B, Gulcehre C, Bahdanau D, Bougares F, Schwenk H, et al. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In: Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing; 2014. pp. 1724–1734. doi: 10.3115/v1/D14-1179

48. Hochreiter S, Schmidhuber J. Long short-term memory. Neural Comput. 1997;9(8):1735–1780. doi: 10.1162/neco.1997.9.8.1735

49. Pedregosa F, Varoquaux G, Gramfort A, Michel V, Thirion B, Grisel O, et al. Scikit-learn: machine learning in Python. J Mach Learn Res. 2011;12:2825–2830.

50. Lian W, Guerra-Manzanares A. MI^{2}DAS: a multi-layer intrusion detection framework with incremental learning for securing industrial IoT networks. arXiv:2602.23846 [Preprint]. 2026. doi: 10.48550/arXiv.2602.23846

Fig11

Click here to access/download;Manuscript;Fig11.tif

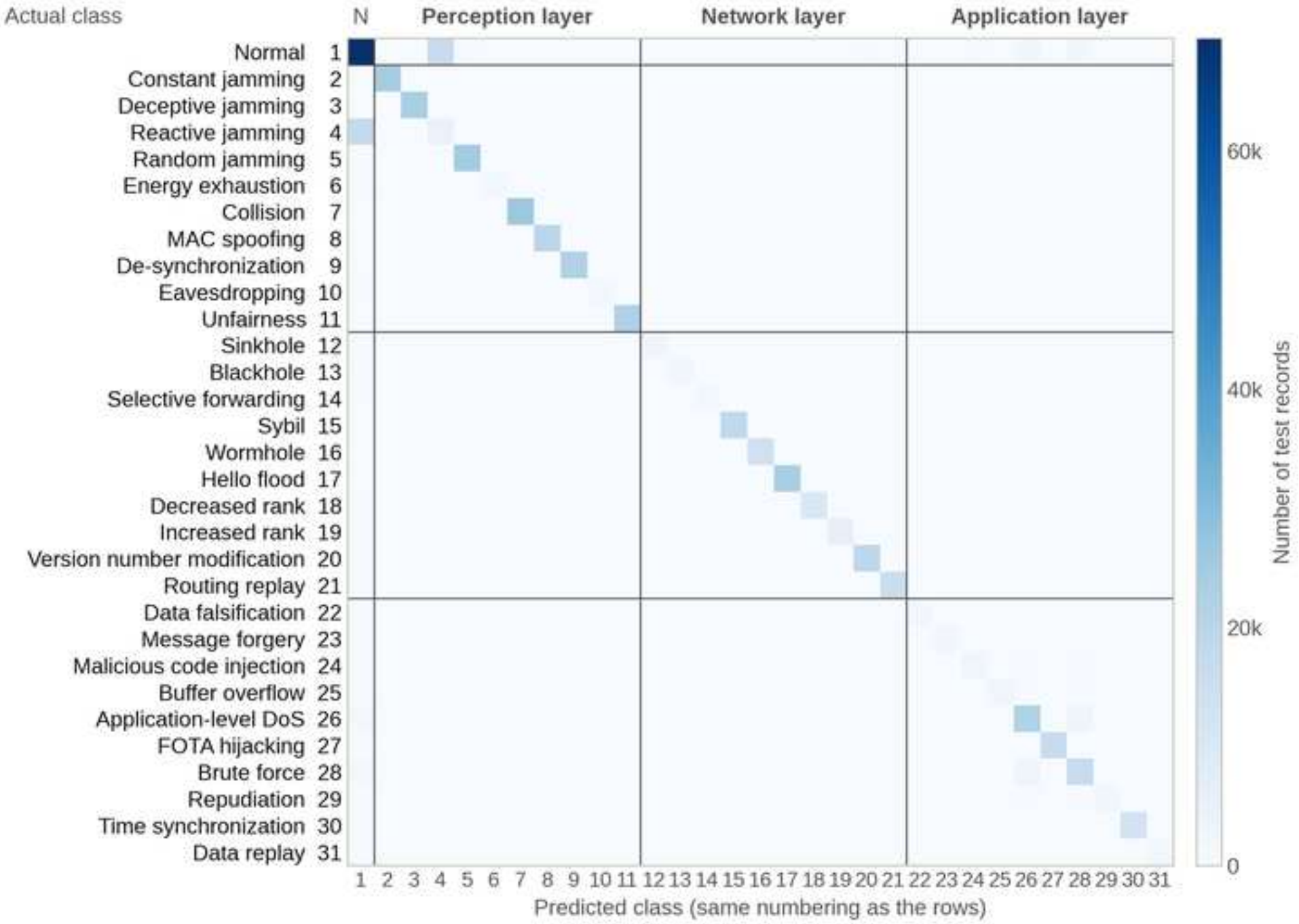


Fig1

Click here to access/download;Figure;Fig1.tif

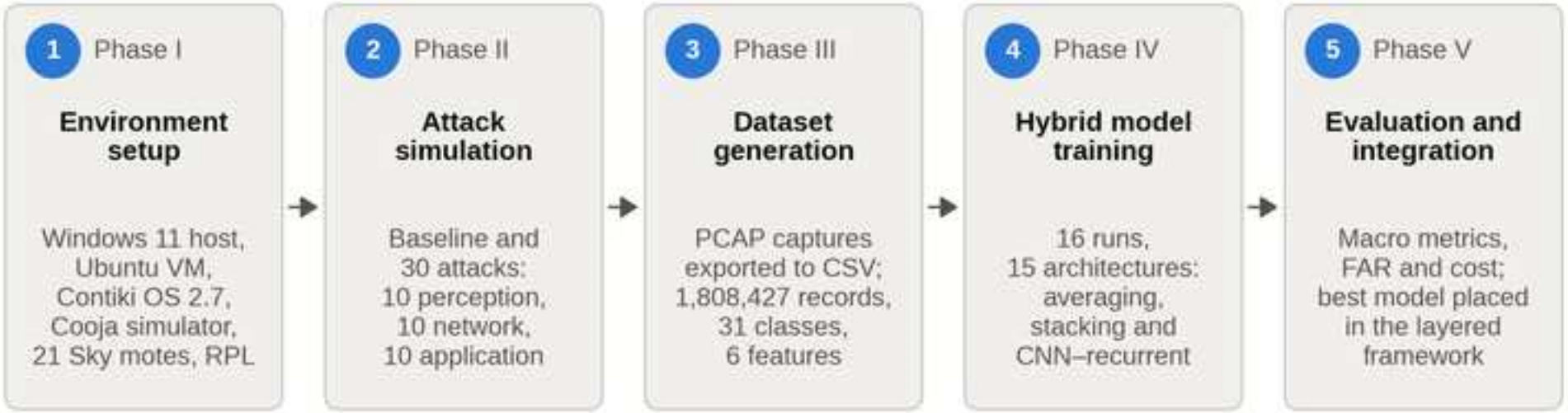


Fig10

Click here to access/download;Figure;Fig10.tif

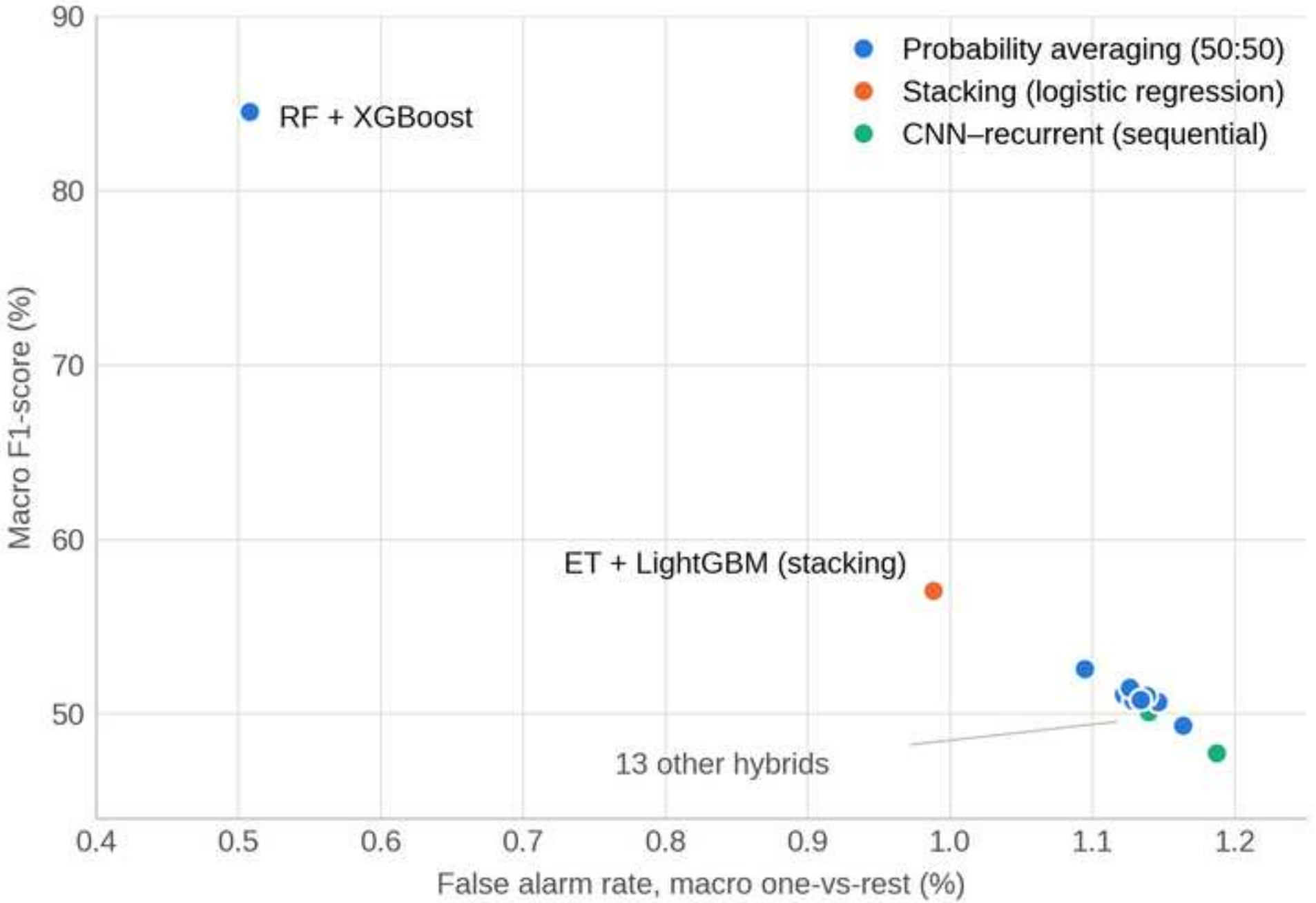


Fig12

Click here to access/download;Figure;Fig12.tif

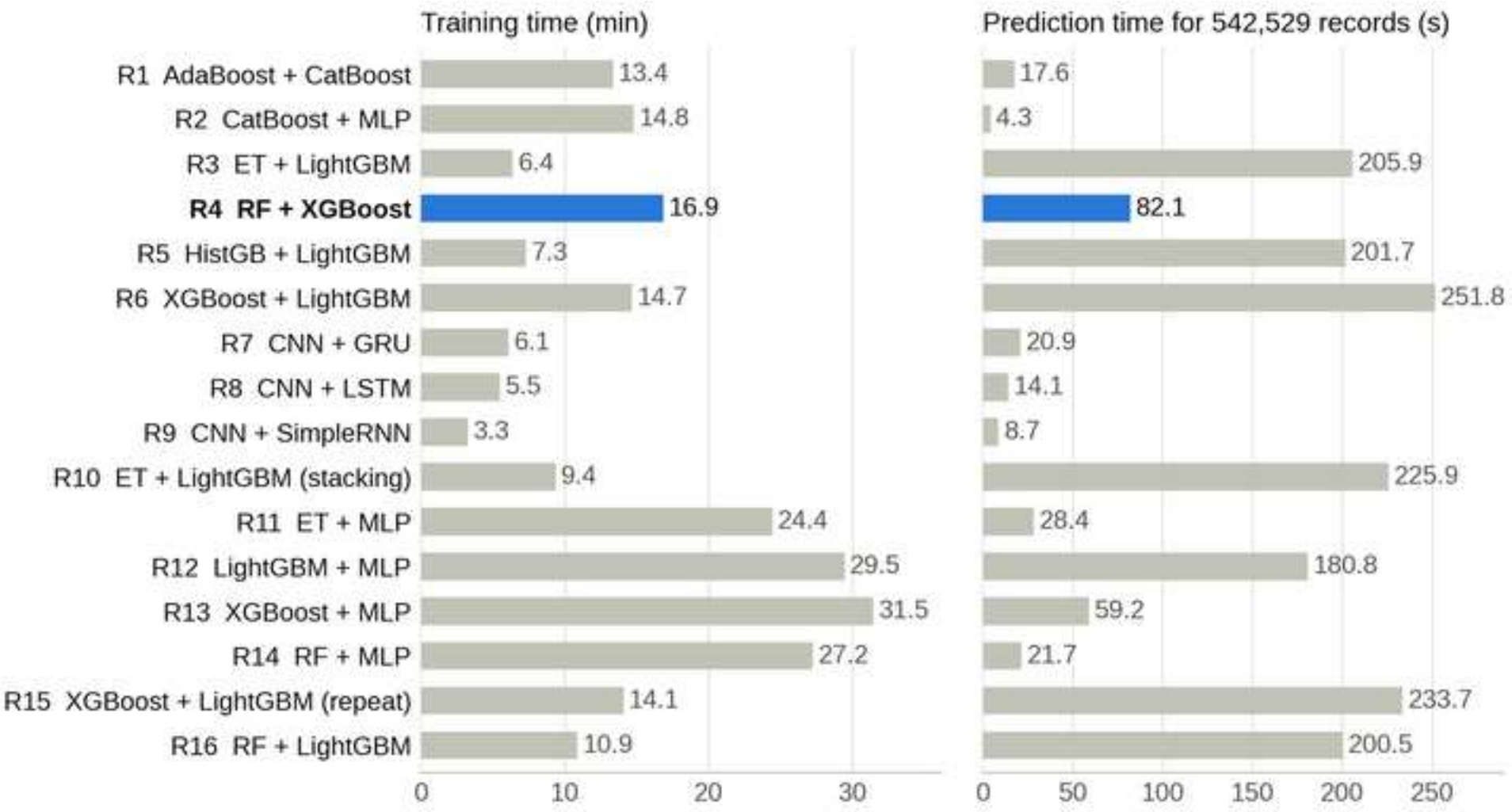


Fig13

Click here to access/download;Figure;Fig13.tif

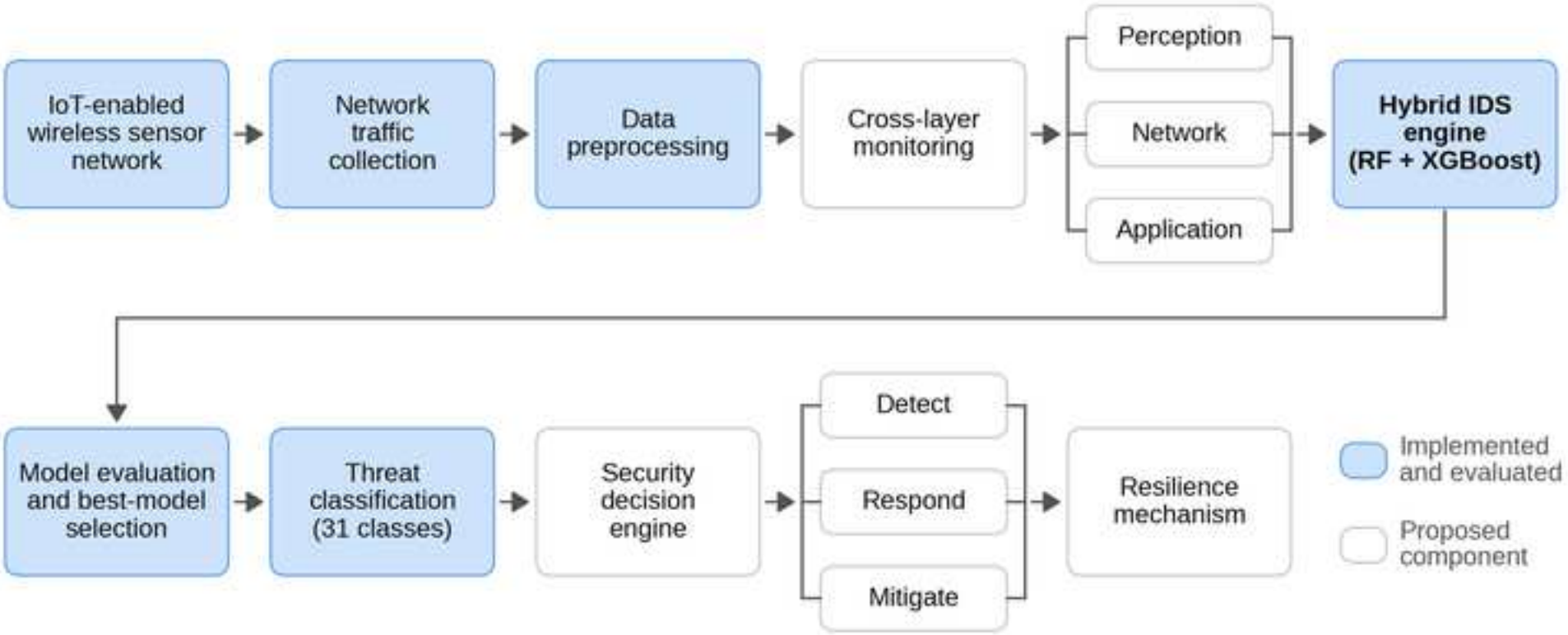


Fig2

Click here to access/download;Figure;Fig2.tif

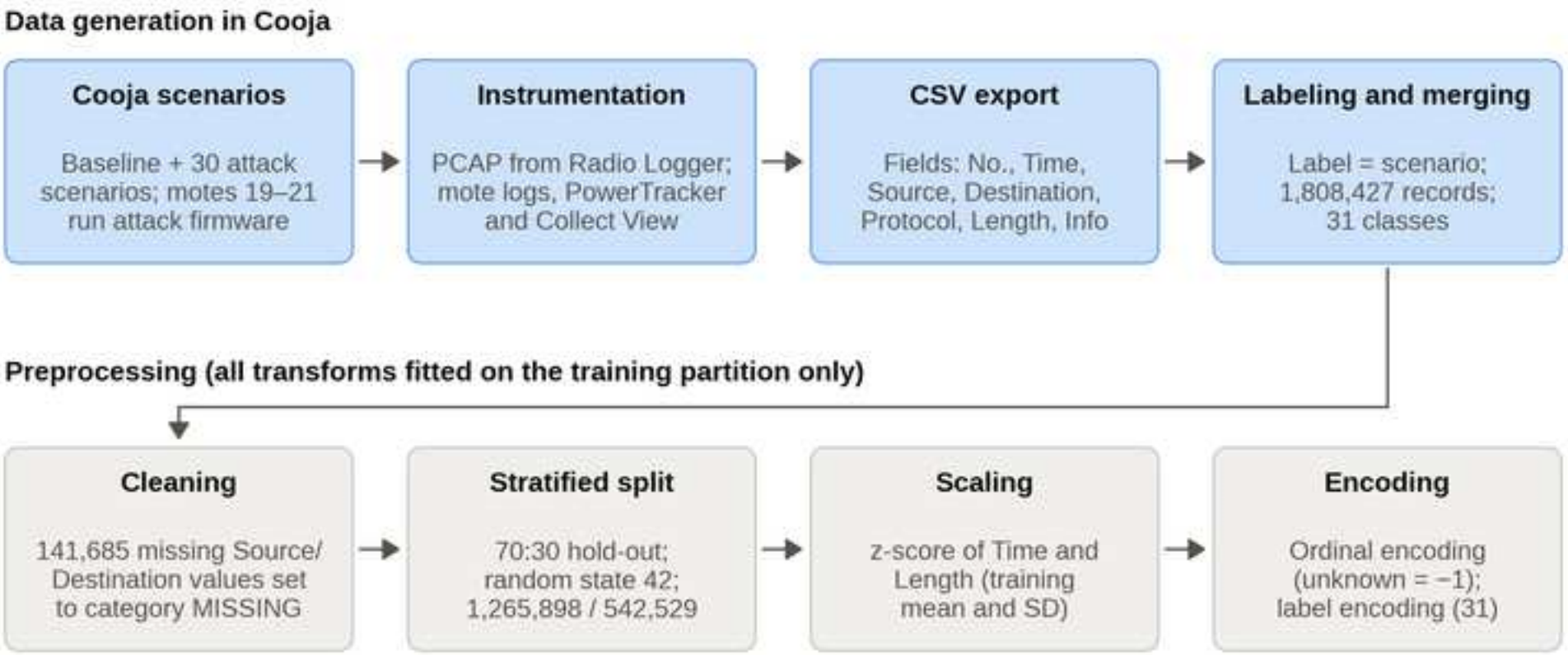


Fig3

Click here to access/download;Figure;Fig3.tif

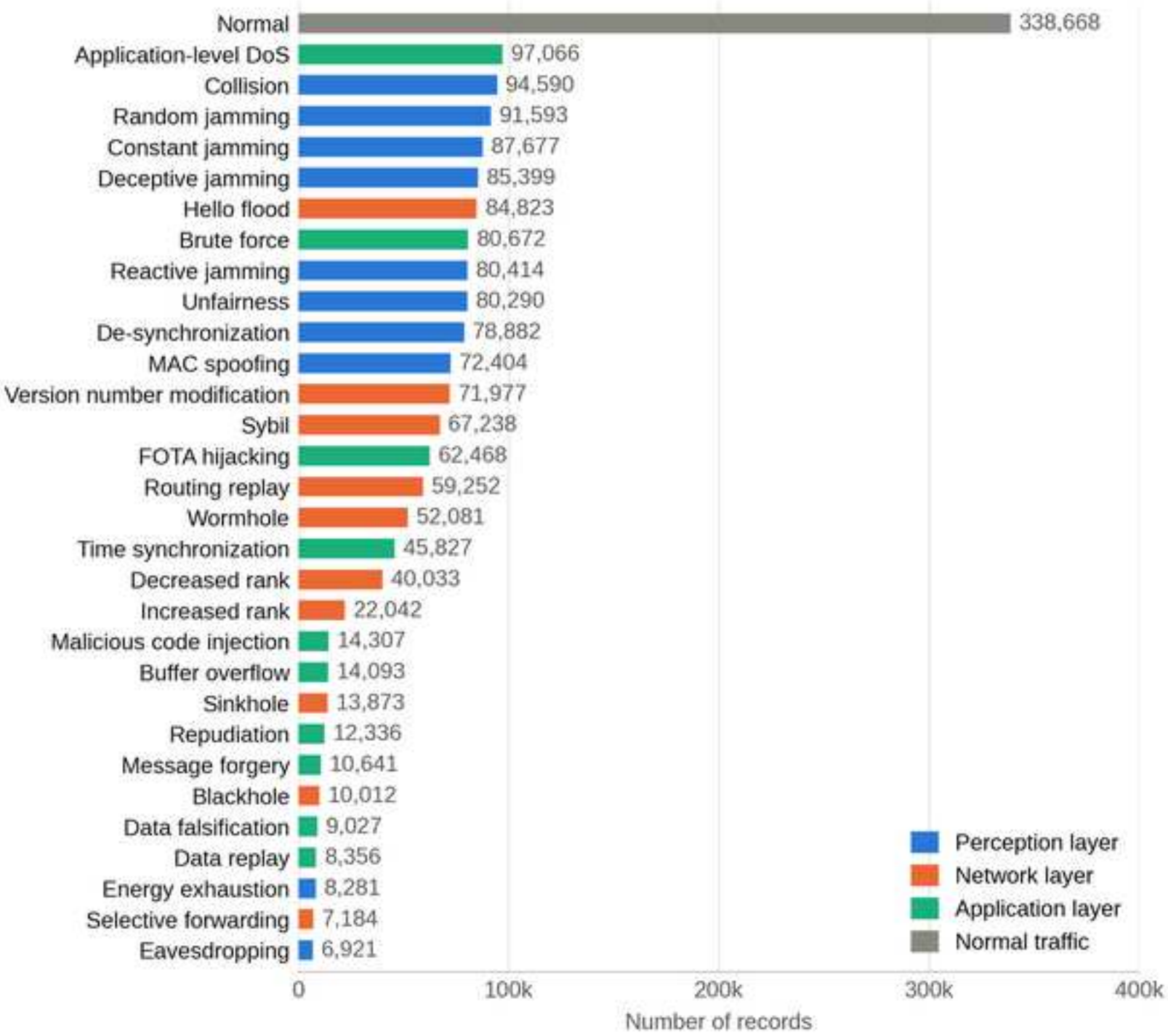


Fig4

Click here to access/download;Figure;Fig4.tif

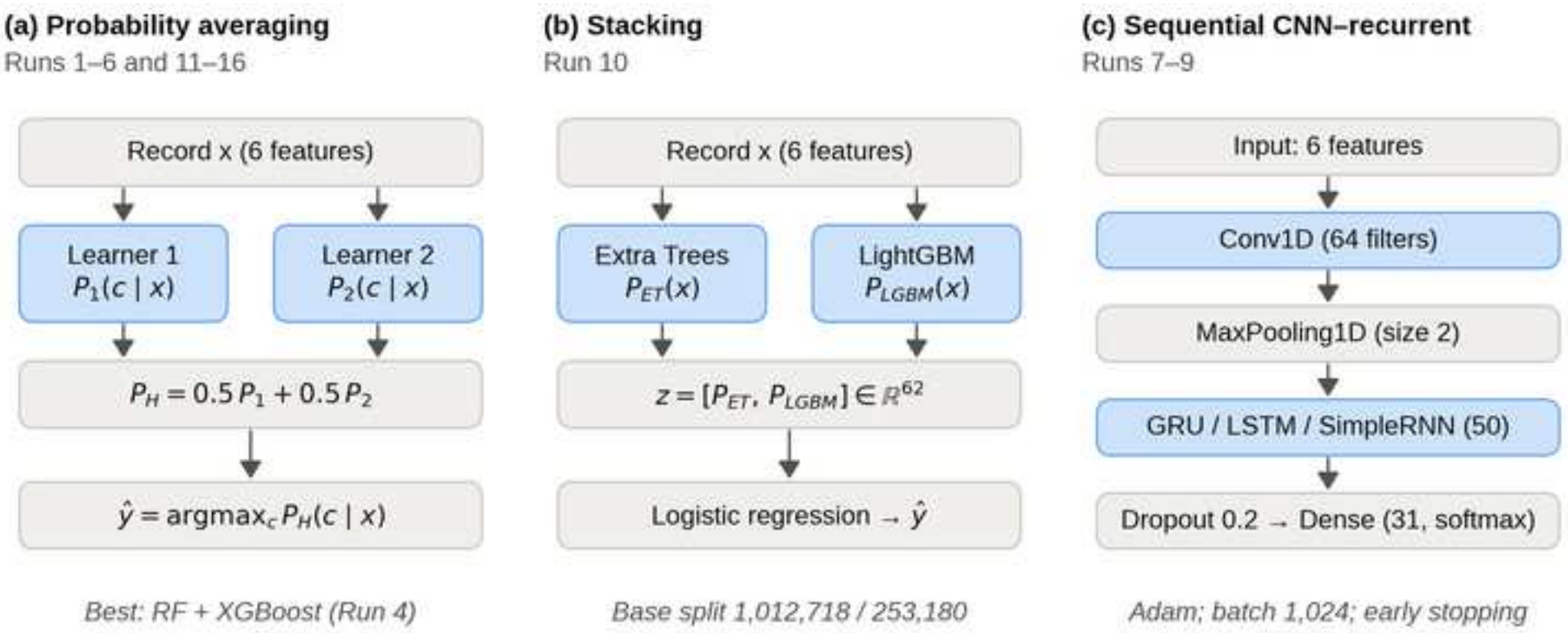


Fig5

Click here to access/download;Figure;Fig5.tif

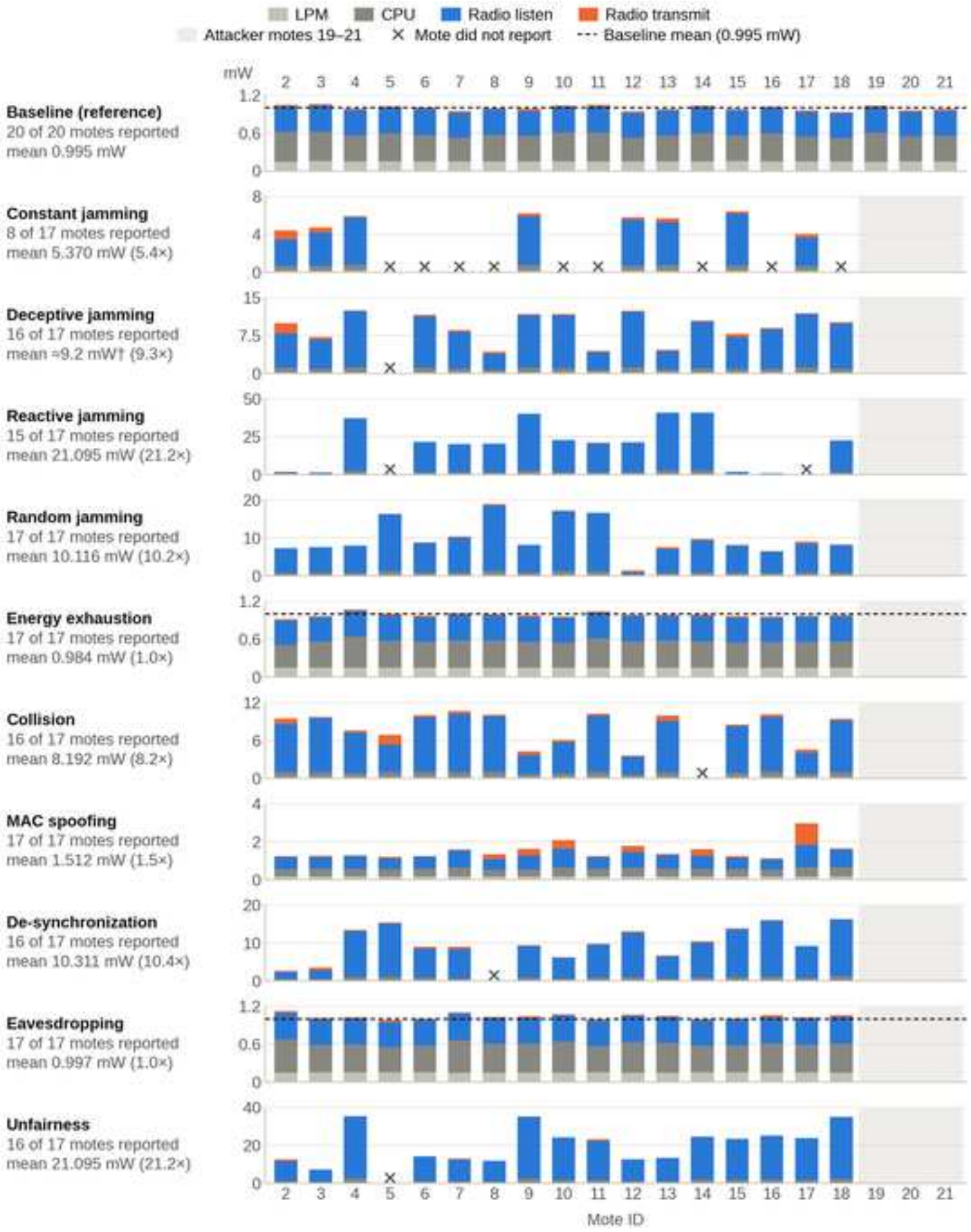


Fig6

Click here to access/download;Figure;Fig6.tif

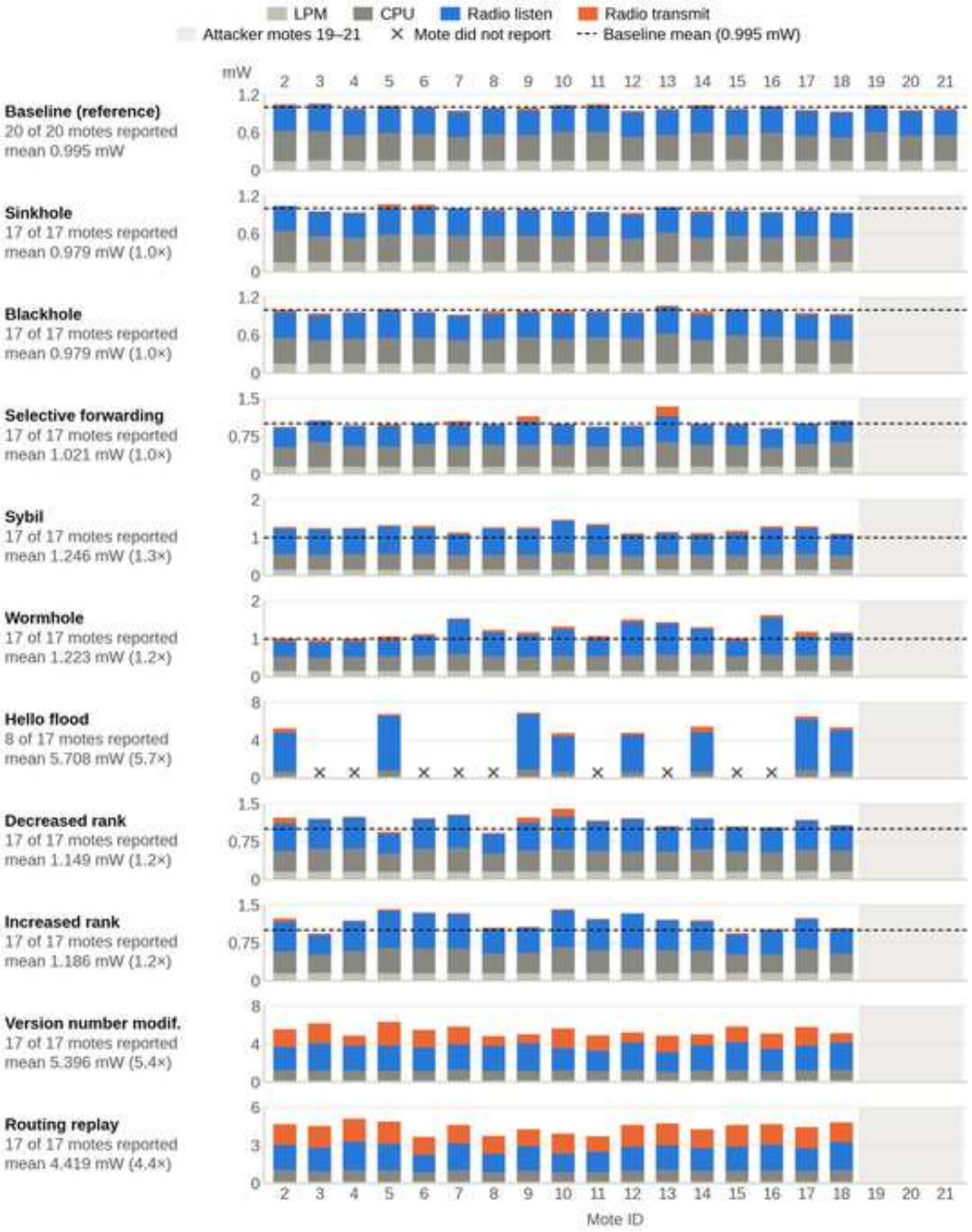


Fig7

Click here to access/download;Figure;Fig7.tif

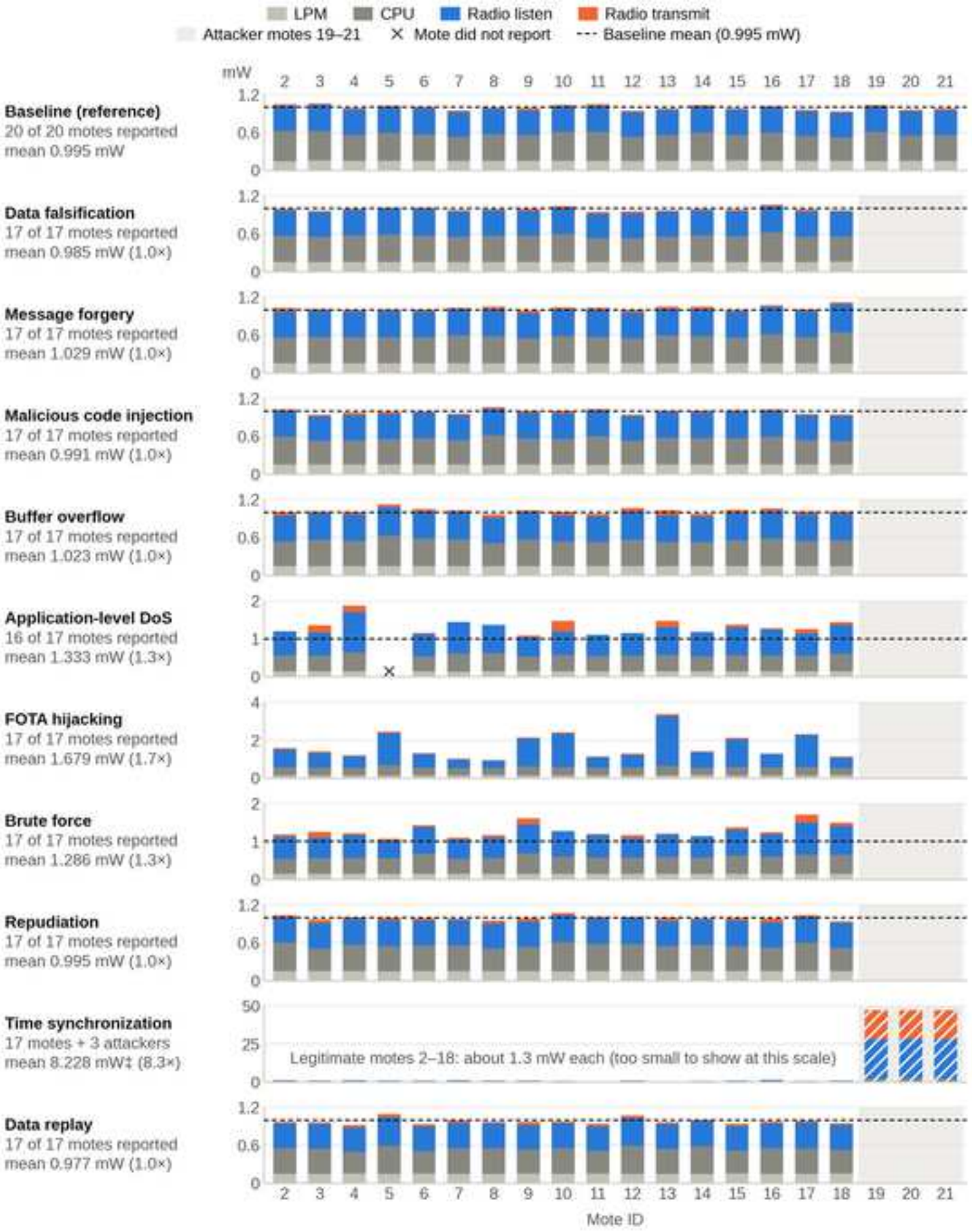


Fig8

Click here to access/download;Figure;Fig8.tif

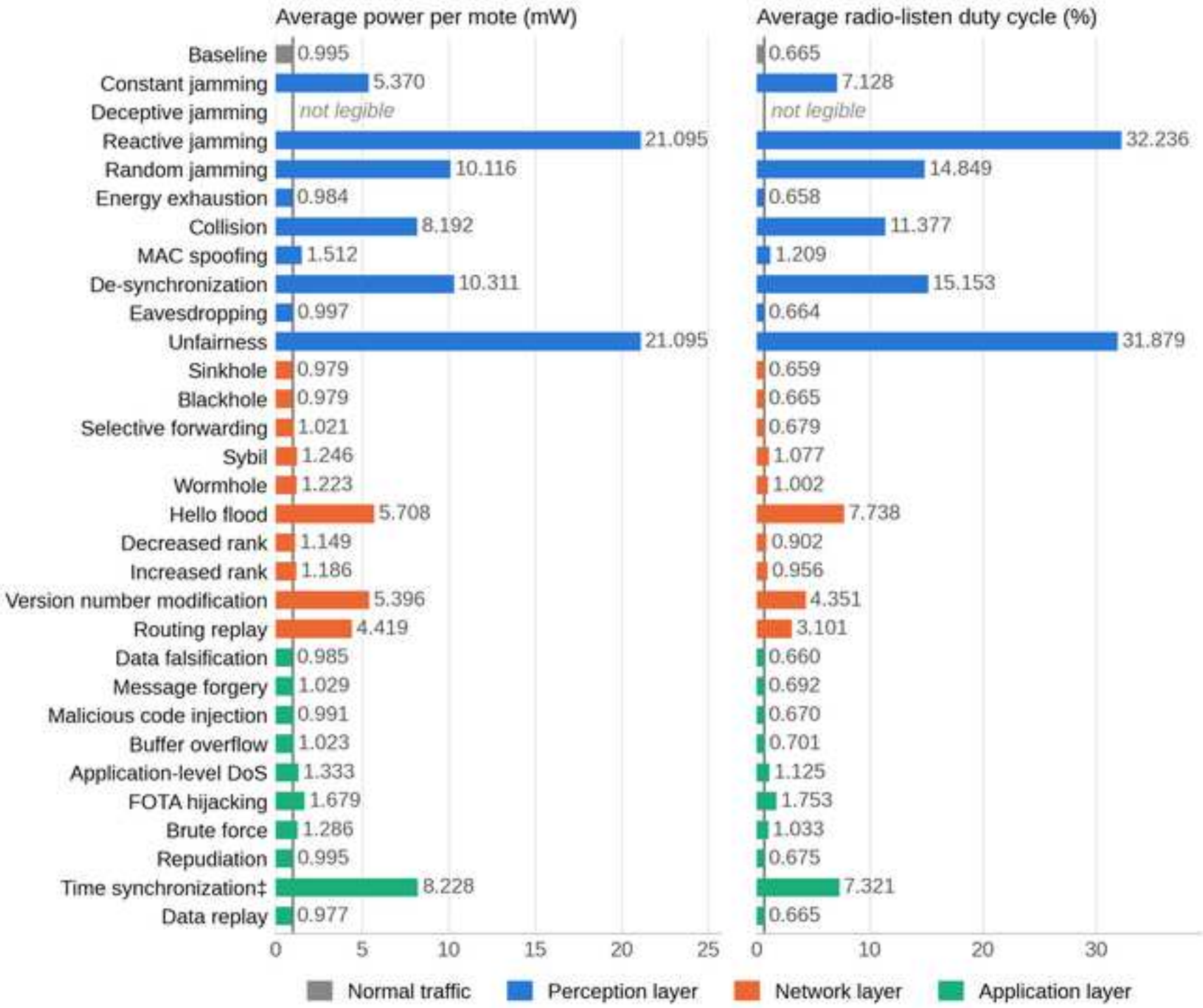


Fig9

Click here to access/download;Figure;Fig9.tif

